

Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

Richard Karlson

April 29, 2025

Hardware Acceleration of Number Theoretic Transform Operations
for Post-Quantum Cryptography

Supervisor: Dr Tomasz Kazmierski

Examiner: Professor Michael Ng

A project report submitted for the award of
BEng Electrical and Electronic Engineering

UNIVERSITY OF SOUTHAMPTON

ABSTRACT

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES
ELECTRONICS AND COMPUTER SCIENCE

A project report submitted for the award of
BEng Electrical and Electronic Engineering

by Richard Karlson

This report describes the implementation and evaluation of a configurable, 256-coefficient Number Theoretic Transform (NTT) hardware accelerator design. The accelerator was implemented using SystemVerilog and synthesised for a Field-Programmable Gate Array (FPGA). A rigid testing strategy and software reference model ensured the correctness of the implementation. This design ultimately achieved typical performance improvements over general hardware, producing a transformation result up to 116 times faster than an Arm Cortex-M3. Despite highly competitive performance, other existing FPGA implementations outperform this design in its current form. However, this project's design shows promising initial results and provides a strong foundation for future work.

Statement of Originality

- I have read and understood the [ECS Academic Integrity](#) information and the University's [Academic Integrity Guidance for Students](#).
- I am aware that failure to act in accordance with the [Regulations Governing Academic Integrity](#) may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have used a section of code originally authored by Professor Mark Zwolinski in the sevenseg.sv file.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell glofficer@ecs.soton.ac.uk

Contents

1	Introduction	1
1.1	Goals	1
2	Background	3
2.1	Post-Quantum Cryptography	3
2.2	Convolutions	4
2.2.1	Wrapped Convolutions	4
2.3	Number Theoretic Transforms	5
2.3.1	Positive and Negative-wrapped NTT and INTT	7
2.3.2	Cooley-Tukey and Gentleman-Sande Algorithms	8
2.4	NTT Hardware Accelerators	10
2.4.1	FPGA vs ASIC vs GPU Accelerators	10
2.4.2	FPGA-Based NTT Accelerators	10
2.4.2.1	Butterfly Unit	11
2.4.2.2	Memory	11
2.4.2.3	Control Unit	11
2.4.2.4	Common Optimisation Techniques	11
2.4.2.5	Pipelined vs Iterative Designs	13
3	Design	15
3.1	Specification	15
3.2	Architecture Overview	16
3.3	Key Components and Design Choices	17
3.3.1	Butterfly Unit	17
3.3.2	Starter Section: Pipelined Processing	19
3.3.3	Engine Section: Iterative Processing	20
3.3.4	Exhaust Section: Final Transformation	21
3.3.5	Control Unit	22
4	Implementation	23
4.1	Development Environment and Tools	23
4.2	SystemVerilog Modules	23
4.3	Butterfly Unit	24
4.4	Memory Architecture	25
4.5	Control Unit	25

4.6	Memory Access Patterns	28
4.7	Twiddle Factor Management	28
5	Testing and Verification	31
5.1	Stage-by-Stage Calculator	31
5.2	Component Testing	33
5.2.1	Butterfly Unit	33
5.2.2	Starter and Engine Sections	34
5.2.3	Starter, Engine, and Exhaust Sections	35
5.3	Full System Testing	35
5.3.1	Parameter Testing	36
5.4	Quartus and FPGA Testing	37
6	Evaluation	39
6.1	FPGA Performance	39
6.2	Performance Comparisons	40
6.2.1	Arm Cortex-M3	40
6.2.2	Other FPGA Accelerator Implementations	40
6.2.2.1	Latency	40
6.2.2.2	Throughput	42
6.2.2.3	Area Usage	43
6.2.2.4	Overall FPGA Comparison	43
6.2.3	Performance Overview	44
6.3	Project Progress	45
6.4	Project Management	45
6.4.1	Time Management	45
6.4.2	Risk Management	46
7	Conclusions	49
7.1	Future Work	50
	Bibliography	51
	Appendix A - Project Management	55
	Appendix B - State Machine Charts	61
	Appendix C - Design Archive	67
	Appendix D - Original Project Brief	69

Chapter 1

Introduction

Current cryptography methods keep data and devices secure against most threats, however advances in quantum computing are forcing cryptography into a post-quantum era. New encryption and decryption methods are more resilient to quantum computers but introduce additional computational demands. The multiplication of polynomials using Number Theoretic Transforms (NTT) is one of these new demands that is used by prominent Post-Quantum Cryptography (PQC) schemes. Unfortunately, this process is resource intensive which slows down encryption and decryption processes, particularly affecting resource-constrained environments.

A solution is a dedicated hardware accelerator for polynomial multiplication and NTT operations. These operations are suitable for parallel execution and require unique mathematical calculations which makes specialised hardware an appealing option. This project aims to investigate the performance improvements of a custom hardware accelerator for NTT operations.

1.1 Goals

- Synthesise a working, configurable hardware accelerator for NTT operations onto an FPGA using SystemVerilog.
- Write a test case program for a CPU to utilise the hardware accelerator.
- Write an equivalent software implementation to run on the CPU without the accelerator.

- Compare the performance between the accelerated and non-accelerated implementations, showing any performance improvements.
- **Stretch Goal:** Implementation of a full encryption and decryption test case with a PQC scheme that utilises NTT and the accelerator.

Chapter 2

Background

This background and literature review aims to provide the necessary information to understand the context, goals, and design decisions made in this project. The review will provide a brief introduction to PQC, the use of NTTs in this field, and NTT hardware accelerators. The information provided is, in most cases, only sufficient for the scope of this project, but further reading is available via the provided references.

2.1 Post-Quantum Cryptography

Traditional cryptography schemes currently keep data safe and secure on devices across the globe and throughout the internet. Unfortunately, the increases in quantum computing capabilities are threatening this paradigm. Current cryptography schemes are based on mathematical problems that are practically unsolvable on the most powerful supercomputers. However, quantum computers will be able to solve these mathematical problems easily due to their unique computing methods [1]. Cryptographers and engineers have foreseen this and have already begun the study of post-quantum cryptography, cryptography methods designed to be resistant to both traditional and quantum computers.

The most promising post-quantum methods rely on lattice-based cryptography, a form of cryptography involving mathematical lattice problems that even quantum computers find difficult to solve [2]. Three of the four standardised finalists in the US National Institute of Standards and Technology (NIST) PQC project rely on lattice-based cryptography. These three are namely Kyber, Dilithium, and

Falcon. Although knowledge of lattice-based cryptography is not necessary for this project, it is important to know that modular polynomial multiplication is the key operation performed in these schemes.

Unfortunately, the requirement for modular polynomial multiplication acts as a significant limitation on performance [3]. Polynomial multiplication algorithms are computationally expensive and scale poorly with higher degrees of polynomial, often scaling with $O(n^2)$ with n as the number of coefficients of the polynomial. One method to counteract this limitation is to use NTTs. However, convolutions must be discussed first in order to understand the effectiveness of NTTs.

2.2 Convolutions

Convolution can be described as the product of two functions and, in the case of this report, the two functions are both polynomials [4]. The convolution of these two polynomials is equivalent to standard polynomial multiplication and can be expressed as

$$Y(x) = A(x) * B(x) \tag{2.1}$$

where $Y(x)$ is the output polynomial, $A(x)$ and $B(x)$ are input polynomials, and $*$ represents convolution. Modular polynomial multiplication can be represented in the same way except a modulo operation would need to be performed on every coefficient of the output polynomial. The number of coefficients in the result, $Y(x)$, will be greater than the number of coefficients in each input polynomial. This is because the resulting polynomial of a standard polynomial multiplication is often a higher order than either of the input polynomials. However, having an output polynomial with more coefficients than the inputs can be an undesirable outcome and thus positive and negative-wrapped convolutions can be used instead.

2.2.1 Wrapped Convolutions

Positive and negative-wrapped convolutions restrict the output polynomial of a normal convolution to have the same number of coefficients as the input polynomials. This is achieved by performing a modulo operation on the convolution result, causing the coefficients of higher-degree terms to be wrapped around and combined with the coefficients of lower-degree terms. The distinction between

positive and negative wrapping is made by the choice of modulo value and how the coefficients are combined during the wrapping process.

The Positive-Wrapped Convolution (PWC) can be represented as

$$PWC = Y(x) \mod (x^n - 1) \quad (2.2)$$

where $Y(x)$ is the result of the convolution of $A(x)$ and $B(x)$ in Equation 2.1, n is the number of coefficients in $A(x)$ and $B(x)$, and $(x^n - 1)$ is the modulo value [3]. The modulo value of $(x^n - 1)$ ensures that all exponents of x in the output polynomial are lower than n by essentially performing modulo n on all exponents. Therefore, coefficients corresponding to values of x with exponents greater than or equal to n are wrapped around and added to the other coefficients, restricting the number of coefficients in the output polynomial.

Similarly, the Negative-Wrapped Convolution (NWC) can be represented as

$$NWC = Y(x) \mod (x^n + 1) \quad (2.3)$$

where $Y(x)$ and n are defined as in Equation 2.2 and $(x^n + 1)$ is the modulo value [3]. The modulo value of $(x^n + 1)$ has the same effect as the $(x^n - 1)$ value used in positive wrapping except for how coefficients are treated when wrapping around. The key difference between positive and negative wrapping is that negative wrapping is subtracting, rather than adding, the coefficients that are being wrapped around. An example of these wrapping methods and the differences between them is shown in Figure 2.1. NTTs can achieve the same result as positive and negative-wrapped convolutions more efficiently by exploiting the convolution theorem.

2.3 Number Theoretic Transforms

As a specific form of Discrete Fourier Transforms (DFTs), NTTs utilise the convolution theorem to perform Equations 2.2 and 2.3 using transforms. In the context of NTTs, the convolution theorem can be expressed as

$$C(x) = INTT(NTT(A(x)) \circ NTT(B(x))) \quad (2.4)$$

where $C(x)$ is the output polynomial, $A(x)$ and $B(x)$ are input polynomials, $\circ$ is an element-wise multiplication, and NTT and $INTT$ represent the forward NTT

$A(x) = 1x^0 + 2x^1 + 3x^2 + 4x^3$ $B(x) = 5x^0 + 6x^1 + 7x^2 + 8x^3$ $n = 4$ $Y(x) = A(x) * B(x)$ $Y(x) = 5x^0 + 16x^1 + 34x^2 + 60x^3 + 61x^4 + 52x^5 + 32x^6$	
PWC	NWC
$Y(x) = 5x^0 + 16x^1 + 34x^2 + 60x^3 + 61x^4 + 52x^5 + 32x^6$	$Y(x) = 5x^0 + 16x^1 + 34x^2 + 60x^3 + 61x^4 + 52x^5 + 32x^6$
<i>Perform modulo 4 operation to all exponents</i>	<i>Perform modulo 4 operation to all exponents</i>
$PWC = 5x^0 + 16x^1 + 34x^2 + 60x^3 + 61x^0 + 52x^1 + 32x^2$	$NWC = 5x^0 + 16x^1 + 34x^2 + 60x^3 - 61x^0 - 52x^1 - 32x^2$
<i>Add coefficients of equal exponent</i>	<i>Subtract coefficients of equal exponent</i>
$PWC = 66x^0 + 68x^1 + 66x^2 + 60x^3$	$NWC = -56x^0 - 36x^1 + 2x^2 + 60x^3$

FIGURE 2.1: Calculation showing the differences between positive and negative wrapping for a polynomial multiplication example. Adapted from [3]

and Inverse NTT (INTT) respectively [3]. The specific NTT implementation of this theorem is expressed in Figure 2.2.

Using NTTs reduces the computational burden of the operation as there is now a simple element-wise multiplication, rather than a full convolution of all the coefficients. However, this process now involves computationally expensive transforms. Fortunately, these transforms have useful mathematical similarities to Fast Fourier Transforms (FFTs) that allow for significant optimisation. Firstly, an important distinction between the positive and negative-wrapped forms of NTTs and INTTs must be made.

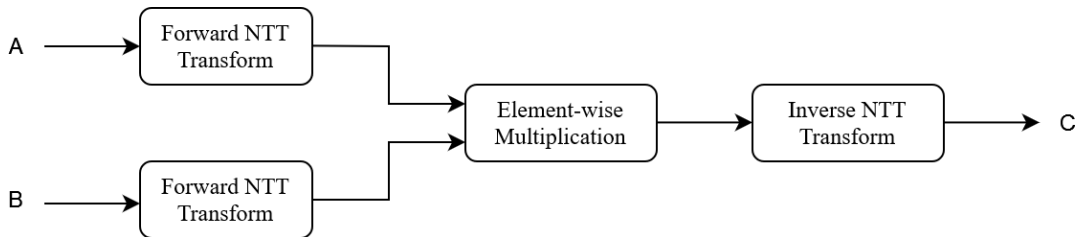


FIGURE 2.2: Flow diagram showing the process of performing a modular polynomial multiplication of A and B using NTT operations with C as the output polynomial.

2.3.1 Positive and Negative-wrapped NTT and INTT

When using positive-wrapped NTT and INTT, the final result of the operation shown in Equation 2.4 is equivalent to the result in Equation 2.2 of a positive-wrapped convolution except a modulo operation has been performed on each coefficient. The same is true when using negative-wrapped NTT and INTT except the result is equivalent to the result in Equation 2.3 of a negative-wrapped convolution.

The choice of positive or negative wrapping is often defined by the cryptographic scheme being used. However, some schemes are not compatible with negative wrapping. This is because wrapping not only affects the form of the final result but also the NTT transform process. Like in FFTs, NTT and INTT operations use a certain coefficient, often called a twiddle factor, during a run of the algorithm. These twiddle factors are different depending on the choice of wrapping.

For positive wrapping, the twiddle factors are defined by the primitive n -th root of unity which can be expressed as

$$\omega^n \equiv 1 \pmod{q} \quad (2.5)$$

and

$$\omega^k \not\equiv 1 \pmod{q} \quad (2.6)$$

where $k < n$ [3]. For negative wrapping, the twiddle factor is the primitive $2n$ -th root of unity which can be defined as

$$\psi^n \equiv -1 \pmod{q} \quad (2.7)$$

and

$$\psi^2 \equiv \omega \pmod{q} \quad (2.8)$$

where ω is the primitive n -th root of unity [3]. The definition for the $2n$ -th root of unity means that some values of q and n have a n -th root of unity but no existing $2n$ -th root of unity. This causes some schemes to only be useable with positive wrapping. The calculation of these twiddle factors is beyond the scope of this project but obtaining the correct values is essential for NTT operations. The use of these twiddle factors is apparent in the NTT algorithms and their optimisations.

2.3.2 Cooley-Tukey and Gentleman-Sande Algorithms

The unoptimised NTT and INTT algorithms scale with $O(n^2)$ which is just as inefficient as non-NTT methods. This is evident in the NTT algorithm, in this case negative-wrapped, which can be represented as

$$\hat{a}_j = \sum_{i=0}^{n-1} \psi^{2ij+i} a_i \mod q \quad (2.9)$$

where $\hat{a}_j$ is the j -th element of the NTT result with $j \in 0, 1, 2, \dots, n-1$ and a_i is the i -th coefficient of the input polynomial [3]. The same can be seen with the similarly defined INTT algorithm represented as

$$a_i = n^{-1} \sum_{j=0}^{n-1} \psi^{-(2ij+i)} \hat{a}_j \mod q \quad (2.10)$$

where a_i is the i -th element of the INTT result with $i \in 0, 1, 2, \dots, n-1$, n^{-1} is a scaling factor, and the inverse twiddle factor, ψ^{-1} , is used [3]. Fortunately, the similarity of NTT operations to FFT operations means that the Cooley-Tukey (CT) and Gentleman-Sande (GS) algorithms can be used to significantly improve the NTT and INTT algorithms respectively.

The CT and GS algorithms use a divide-and-conquer method and rely on the periodic and symmetrical properties of the twiddle factors to break up the NTT and INTT algorithms into smaller independent calculations. Continuing with the negative-wrapped case, an expression for the CT algorithm can be defined as

$$\hat{a}_j = A_j + \psi^{2j+1} B_j \mod q \quad (2.11)$$

$$\hat{a}_{j+n/2} = A_j - \psi^{2j+1} B_j \mod q \quad (2.12)$$

where $A_j = \sum_{i=0}^{n/2-1} \psi^{4ij+2i} a_{2i}$ and $B_j = \sum_{i=0}^{n/2-1} \psi^{4ij+2i} a_{2i+1}$ [3]. Similarly, the GS algorithm can be defined as

$$a_{2i} = (A_i + B_i) \psi^{-2i} \mod q \quad (2.13)$$

$$a_{2i+1} = (A_i - B_i) \psi^{-2i} \mod q \quad (2.14)$$

where $A_i = \sum_{j=0}^{n/2-1} \psi^{-4ij} \hat{a}_j$ and $B_i = \sum_{j=0}^{n/2-1} \psi^{-4ij} \hat{a}_{j+n/2}$ [3]. These equations show the algorithms working with two calculations in parallel and reusing calculated intermediate values. Visually, this can be described in Figure 2.3, showing

the origin of the term "butterfly" for the description of these algorithms.

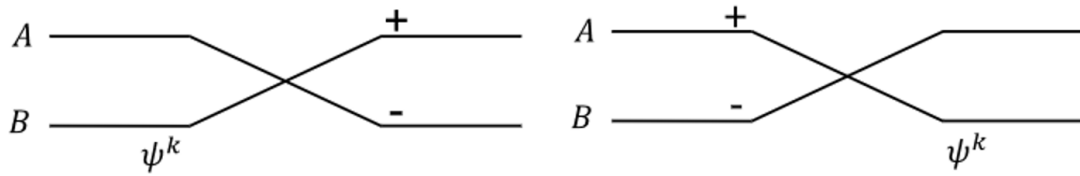


FIGURE 2.3: Line diagrams showing the butterfly-like nature of the CT (left) and GS (right) algorithms. Taken from [3].

The CT and GS algorithms often work through numerous intermediate steps known as stages to reach the final answer. The number of stages required is $\log_2 n$ and a visual representation of these stages can be seen in Figure 2.4. These algorithm optimisations allow the scaling of NTT operations to reach $O(n \log n)$ as opposed to the $O(n^2)$ scaling previously. More importantly, the inherent parallel nature of the CT and GS algorithms makes them particularly appealing for use with dedicated parallel hardware.

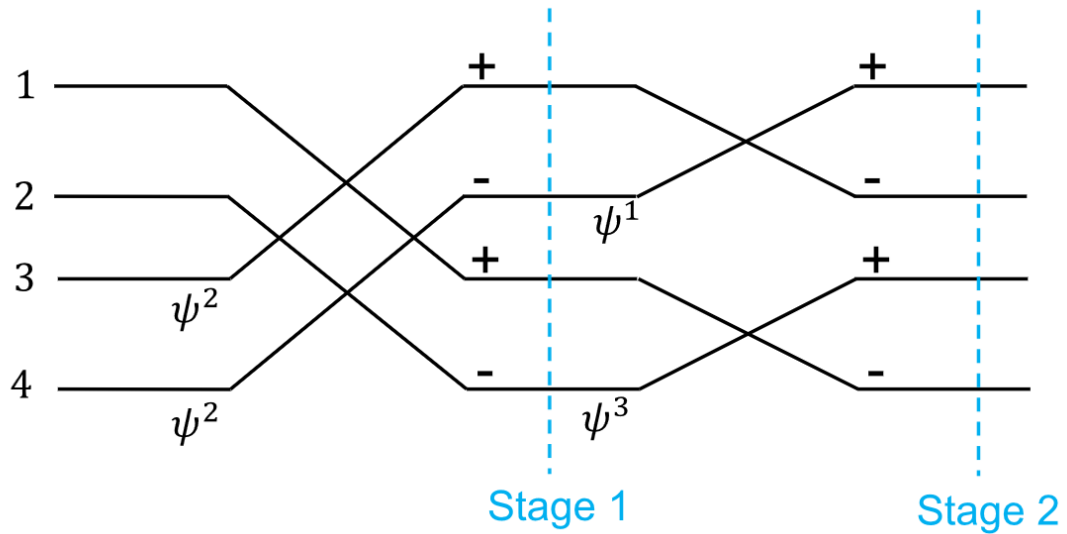


FIGURE 2.4: Line diagrams showing the multiple stages the CT algorithm goes through to reach a final answer with $4x^3 + 3x^2 + 2x + 1$ as the input polynomial, thus $n = 4$. Taken from [3]

2.4 NTT Hardware Accelerators

PQC schemes will need to be deployed on a variety of devices with different performance and power constraints but the significant computational burden of NTT operations makes this challenging. A solution to this is dedicated hardware accelerators for NTT operations. A variety of hardware acceleration technologies have been proposed, including Field-Programmable Gate Array (FPGA), Application Specific Integrated Circuit (ASIC), and Graphics Processing Unit (GPU) implementations.

2.4.1 FPGA vs ASIC vs GPU Accelerators

Each of these technologies has its own advantages and drawbacks when compared to one another, as shown in Table 2.1. Currently, the most common proposed

Metric	GPU	ASIC	FPGA
Performance	High	Very High	Medium
Power Usage	High	Low	Medium
Cost	Medium	Very High	Low
Reconfigurable	No	No	Yes

TABLE 2.1: Comparison between key metrics for GPU, ASIC, and FPGA technologies for hardware acceleration. Adapted from [5].

technology is FPGAs largely due to their reconfigurability, relatively simple design flow, and lower development cost. Although ASICs would perform better than FPGAs, ASICs require a large upfront manufacturing cost and cannot be rapidly reconfigured like FPGAs. GPUs have extensive parallel computing performance but consume significantly more power than ASICs and FPGAs and have no custom hardware configurability. FPGAs also typically use fewer hardware resources than GPUs and can achieve comparable performance [6]. This leaves FPGAs as the technology of choice for NTT hardware accelerators.

2.4.2 FPGA-Based NTT Accelerators

There are a vast number of proposed FPGA-based NTT accelerator designs, each with unique goals and optimisations, but the high-level design of most of these accelerators is largely the same. The main components of a FPGA-based NTT

accelerator are a control unit, a set of memory, and butterfly units. Most implementations use specific configurations of these components but the main features are present in all of them.

2.4.2.1 Butterfly Unit

The butterfly unit is the main processing element of the accelerator. These units are responsible for running the CT and GS butterfly computations as efficiently as possible. The number of inputs and corresponding outputs each unit handles at a time is called the radix of the unit. Using radix-2 butterfly units is common because it allows for greater modularity, is simpler to design, and uses less area to implement while achieving the same transformation result [3].

2.4.2.2 Memory

The inputs, outputs, and twiddle factors of the butterfly units need to be stored throughout the operation, leading to the need for memory. RAMs and ROMs are typically the structures used to provide memory and are particularly advantageous for FPGA implementations where dedicated hardware such as Block RAMs (BRAMs) are already present on the chip.

2.4.2.3 Control Unit

The control unit is the main coordinator of the butterfly units and memory, ensuring the CT and GS algorithms are being run correctly. Through the use of control signals, the control unit primarily ensures that memory access patterns are correct but can have other functions. This unit also acts as an important interface between the accelerator and any external hardware.

2.4.2.4 Common Optimisation Techniques

There exist several optimisation techniques that can be utilised in these accelerators to better performance and resource efficiency. These techniques encounter the known hardware problem of power, performance, and area optimisation where focussing on one is often at the expense of another. Some common techniques used involve:

- **Pipelining**

Using multiple butterfly units in a pipelined configuration enables multiple transformations to be in the stage of processing at once, increasing the total throughput and performance of the system [7].

- **Parallelism**

Multiple butterfly units can be placed in parallel to exploit the inherent parallelism of the CT and GS algorithms. This improves the speed of a single transformation by reducing the number of clock cycles to complete a single transformation. When coupled with pipelining, parallelism can achieve significant performance increases [8].

- **Modular Reduction Algorithms**

An optimised modular reduction method can be used in these accelerators because the modulus value, q , is a known constant value in any PQC scheme. Popular modular reduction techniques, such as the Barrett reduction algorithm, can be used to replace expensive and slow division operations with more efficient multiplication operations [9].

- **Iterative Butterfly Units**

Individual butterfly units can be reused iteratively in place of additional butterfly units. This allows for an area-optimised accelerator at the expense of performance, which may be useful in resource-constrained environments [3]. Using butterfly units iteratively also prevents pipelining due to structural hazards but parallelisation can still be used effectively.

- **Unified Butterfly Units**

Butterfly units can be designed to do both CT and GS computations, reducing the need for separate units for each algorithm. This can significantly reduce the area usage of the accelerator and enable better hardware utilisation by reusing components for both computation modes. A drawback to this optimisation is the increased complexity of the control unit by adding signals to configure the butterfly unit for each mode.

Each available optimisation technique has its own usefulness depending on the implementation use case with many implementations using a combination of techniques.

2.4.2.5 Pipelined vs Iterative Designs

A complicating factor in the design of purely pipelined NTT hardware accelerators is the complex access patterns required by the transformation. As shown in Figure 2.5, a set of butterfly units cannot simply feed their outputs directly into the inputs of the next unit ahead of it. Routing hardware is required to ensure that each butterfly unit receives the correct inputs from the previous stage. The reliance of one butterfly unit input on potentially any of the previous stage's outputs means that you cannot process a select subset of coefficients at a time, as eventually a coefficient from outside the subset will be needed. This creates a requirement for enough butterfly units in parallel to calculate an entire stage at a time.

A pipelined accelerator designed for 8-coefficient polynomials, like Figure 2.5, will therefore need 4 parallel butterfly units and 3 stages of these units, making a total of 12 required butterfly units. The number of required butterfly units grows quickly as the number of coefficients increases. For example, a 256-coefficient purely pipelined design requires 128 parallel units with 8 stages, making a total of 1024 individual butterfly units. While purely pipelined designs offer significant throughput advantages, the number of butterfly units required will often take up too much area and resources for many application scenarios. This explains why some designs choose a more iterative design instead.

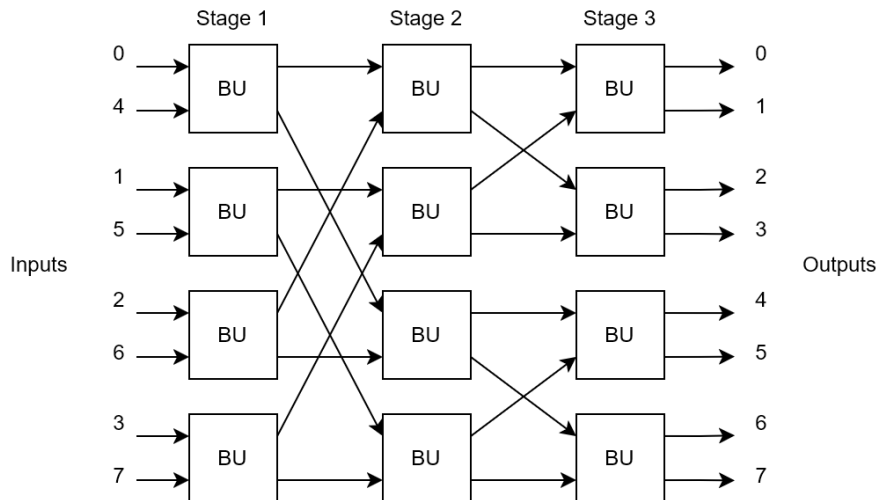


FIGURE 2.5: An example of a pipelined NTT hardware accelerator with twelve butterfly units processing an 8-coefficient polynomial, showing the flow of each coefficient index through the units.

Iterative designs, as shown in Figure 2.6, reuse the same butterfly units over and over again in place of having additional units. A set of memory stores outputs and provides the correct inputs to the units for each step of computation. This set of

memory uses complicated access patterns to provide the correct inputs and ensure data that still needs to be processed is not overwritten. These designs can be very compact with only one butterfly unit running 1024 times to complete the work of 1024 separate units. One clear drawback to this approach is the lack of throughput as only one transformation operation can be using the hardware resources at any given time. The design used in this project explores a hybrid approach, exploring the intersection of a pipelined and iterative design.

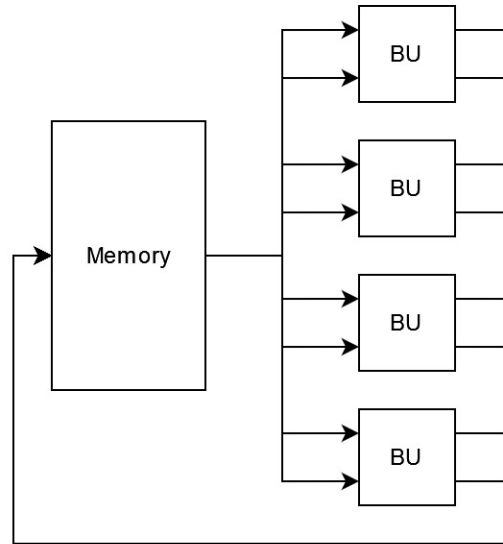


FIGURE 2.6: An example of an iterative NTT hardware accelerator with four butterfly units iteratively processing eight coefficients on each cycle.

Chapter 3

Design

3.1 Specification

To be effective in any real-world applications, an NTT accelerator must support at least one PQC scheme. The Kyber scheme is the least demanding of the previously mentioned schemes, requiring just 256 coefficients and a modulo value of 7681 (version 1) [10]. Given its relatively light computational requirements, Kyber scheme support is a suitable goal for this accelerator design.

Supporting Kyber ensures that this design can be benchmarked against existing accelerators and allows for direct performance comparisons. To be a viable choice, the accelerator must demonstrate an advantage in at least one performance metric such as throughput, latency, or area usage over existing implementations. This one performance advantage could potentially make the accelerator useful in a certain application scenario. An example of such a scenario could be an embedded system where area and power usage must be as low as possible, at the expense of speed.

As polynomial multiplication is fundamental to PQC schemes, the accelerator must be able to perform both forward and inverse NTT transformations. The proposed design aims to achieve this by exploring a hybrid approach, combining elements of both pipelined and iterative designs. The following sections describe the architecture, individual components, and the reasoning behind certain design decisions.

3.2 Architecture Overview

The high-level design of this Kyber-compatible accelerator, shown in Figure 3.1, is structured into three main sections, each responsible for different stages of the transformation process. As mentioned in Chapter 2, a 256-coefficient polynomial will require eight stages of transformation, as $\log_2(256) = 8$. The "starter" section processes the initial three stages using a pipelined approach to improve throughput. The "engine" section performs stages four through seven with an iterative approach to minimise area usage. Finally, the "exhaust" section completes the final stage and applies scaling where necessary, outputting fully transformed results.

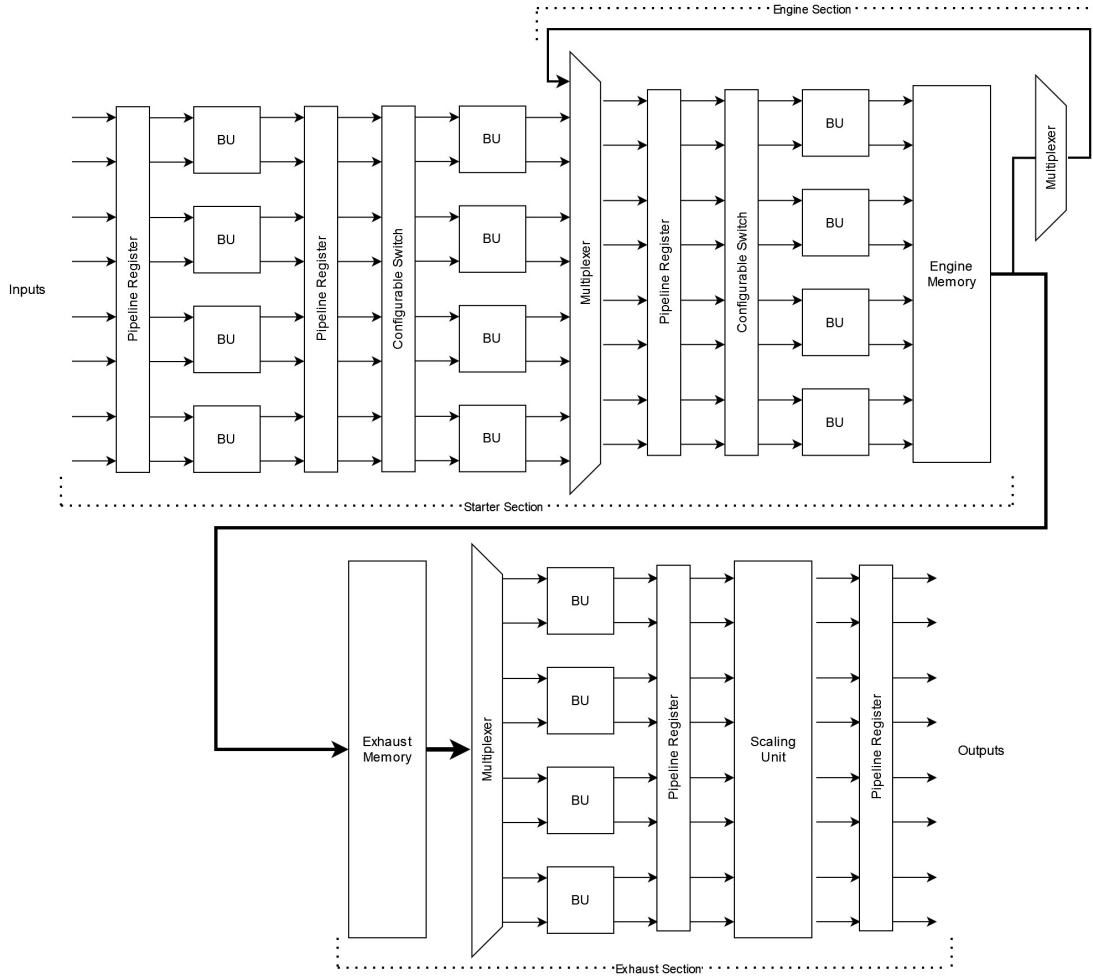


FIGURE 3.1: Block diagram representing the high-level design of the NTT hardware accelerator, with each section of the design shown.

This hybrid design seeks to achieve the best of both design types. A fully pipelined approach would maximise throughput but at the cost of higher resource usage,

while a purely iterative approach would minimise hardware costs but reduce overall performance. By breaking up the design into different sections, the design should achieve higher throughput than a purely iterative approach while consuming less area than a purely pipelined design.

The accelerator design ultimately implements an NTT or INTT algorithm, making the choice of algorithm vitally important. The algorithms used for the NTT and INTT transformations, shown in Algorithms 1 and 2 respectively, are the algorithms used with the CoHA-NTT accelerator, a different NTT accelerator design and implementation [11]. These algorithms were chosen due to their simplicity and ease of implementation. Although these two algorithms theoretically allow either positive or negative-wrapping to be used, the design in this project is focussed on the negative-wrapped version as the more common choice in the context of PQC [3]. The element most affected by the choice of algorithm is the butterfly unit as it bears most of the computation burden.

Algorithm 1 Forward NTT transformation algorithm. Adapted from [11]

```

1:  $k = 1$ 
2:  $l = (n/2)$ 
3:  $v = \log_2(n)$ 
4: while ( $l \geq 1$ ) do
5:   for ( $s = 0; s < n; s = j + l$ ) do
6:      $z = \omega^{br(k,v)} \bmod q$ 
7:      $k = k + 1$ 
8:     for ( $j = s; j < (s + l); j++$ ) do
9:        $c = a[j + l] \times z \bmod q$ 
10:       $a[j + l] = a[j] - c \bmod q$ 
11:       $a[j] = a[j] + c \bmod q$ 
12:    end for
13:  end for
14:   $l = l/2$ 
15: end while
16: return  $a$ 
```

3.3 Key Components and Design Choices

3.3.1 Butterfly Unit

At the centre of the design is the butterfly unit, responsible for the primary computations of both NTT and INTT operations. The butterfly unit has been designed

Algorithm 2 Inverse NTT transformation algorithm. Adapted from [11]

```

1:  $k = 0$ 
2:  $l = 1$ 
3:  $v = \log_2(n)$ 
4: while ( $l \leq (n/2)$ ) do
5:   for ( $s = 0; s < n; s = j + l$ ) do
6:      $z = \omega^{br(k,v)+1} \bmod q$ 
7:      $k = k + 1$ 
8:     for ( $j = s; j < (s + l); j++$ ) do
9:        $c = \bar{a}[j] - \bar{a}[j + l] \bmod q$ 
10:       $\bar{a}[j] = \bar{a}[j] + \bar{a}[j + l] \bmod q$ 
11:       $\bar{a}[j + l] = c \times z \bmod q$ 
12:    end for
13:  end for
14:   $l = l \times 2$ 
15: end while
16: for ( $i = 0; i < n; i++$ ) do
17:    $\bar{a}[i] = \bar{a}[i] \times n^{-1} \bmod q$ 
18: end for
19: return  $\bar{a}$ 

```

as a unified, radix-2 butterfly unit that can perform both the forward and inverse NTT operations. This means the same hardware resources are used to perform lines 9-11 of both Algorithm 1 and Algorithm 2. The unified design allows a single hardware block to handle both operations, reducing area usage and adding flexibility. A radix of two also allows more flexibility as higher radices can be achieved by instantiating multiple units in parallel.

As seen in Figure 3.2, the butterfly unit takes in two integer inputs and a twiddle factor before performing modular arithmetic to produce the two transformed outputs. A control signal determines whether the unit is in NTT or INTT execution mode.

Since there are a total of 16 butterfly units across the entire accelerator, optimising this component is critical for overall efficiency. One of the most computationally expensive tasks in the transformation is the modular reduction. Unoptimised reduction methods use division operations which are costly in both hardware area and latency, so the butterfly unit uses the Barrett reduction algorithm instead. This algorithm, shown in Algorithm 3, replaces division with more efficient shifts and multiplications. This optimisation improves performance while also reducing area usage.

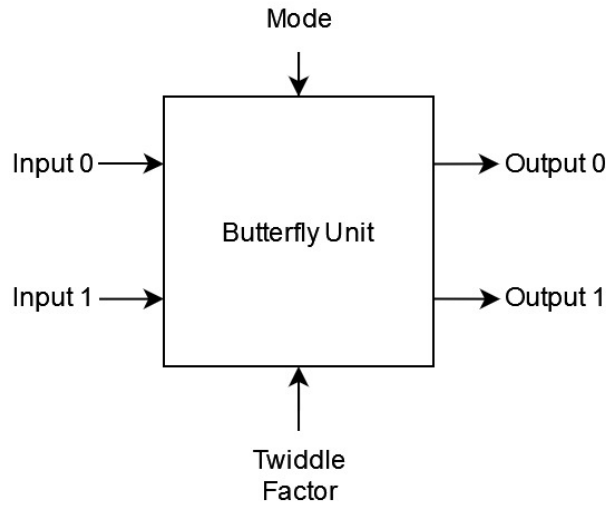


FIGURE 3.2: Block diagram representing the butterfly unit design with all its relevant inputs and outputs.

Algorithm 3 Barrett reduction algorithm for $a \bmod q$. Adapted from [9]

```

1:  $k = \log_2(q)$ 
2:  $m = \lfloor 2^k/q \rfloor$ 
3:  $z = (a \times m) \gg k$ 
4:  $a = a - z \times q$ 
5: if  $a \geq q$  then
6:    $a = a - q$ 
7: end if
8: return  $a$ 

```

3.3.2 Starter Section: Pipelined Processing

The starter section processes the first three stages of the transformation using 12 butterfly units arranged in 3 columns, each containing 4 units. The butterfly units are connected through configurable switches that adjust the data flow based on whether an NTT or INTT operation is being performed.

The configurable switches reorganise the coefficients to match the required access patterns for the next stage. In NTT mode, the switches perform a 4-switch then a 2-switch while conversely, in INTT mode, the switches perform a 2-switch then a 4-switch, as illustrated in Figure 3.3. The naming of these switch operations refers to the maximum number of places a coefficient is moved away from its original position during a switch. A multiplexer before the second switch selects whether to use the outputs from the previous column or the engine section. This multiplexer therefore determines whether the starter or engine section is using the third set of

butterfly units. The output of the final column is written to memory where it is stored for further processing in the engine section.

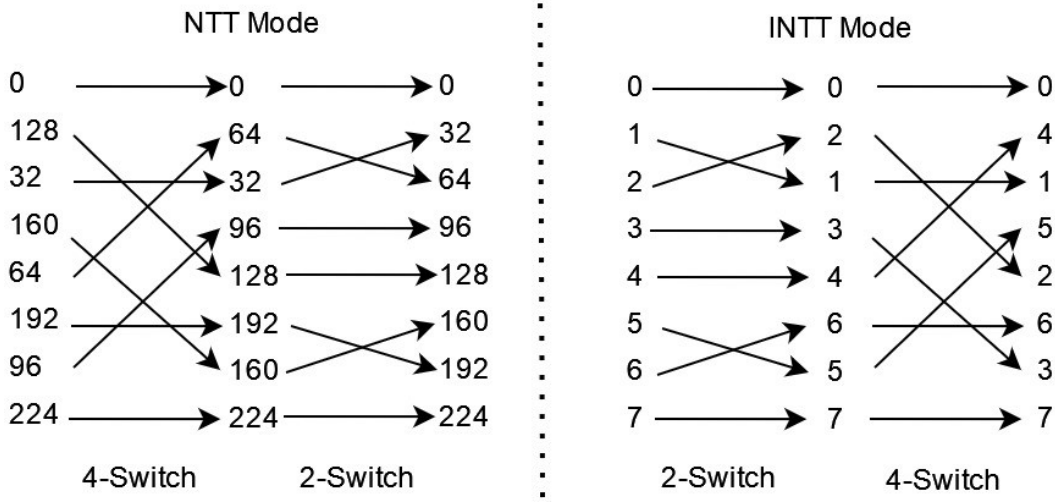


FIGURE 3.3: Diagram showing an example of how coefficients flow in the starter section in both NTT and INTT mode.

The choice to include three pipeline stages was based on a trade-off between throughput and hardware resource usage. Increasing the number of pipelined stages would improve throughput but require significantly more hardware. Each additional pipelined stage doubles the number of units needed in parallel to accommodate larger switches between units. At the same time, reducing the number of pipeline stages would increase reliance on iterative computation, negatively affecting throughput. Three stages were chosen as an optimal balance for experimentation, providing a meaningful performance boost while keeping resource usage at a manageable level for implementation on some FPGAs.

3.3.3 Engine Section: Iterative Processing

After the first three stages have been processed, the next four stages are handled by the engine section, which reuses four butterfly units from the starter section to process data iteratively. This section uses memory to keep reusing the same butterfly units, performing the necessary computations over multiple iterations.

The engine section features a dedicated memory block capable of storing 128 coefficients, allowing half of the total number of coefficients to be in processing at once. A multiplexer selects the appropriate coefficients from memory and routes them to the butterfly units, which then write the computed results back into memory. The process repeats until all coefficients have completed their seventh stage

of transformation. The results are then ready to be transferred to the exhaust section for final transformation.

Many design decisions in the engine section depend on decisions made for the starter section as hardware is reused. However, one key design consideration was determining the size of the memory. Since the engine section must perform all switches other than the 4-switch (starter section), 2-switch (starter section), and 128-switch (exhaust section), the engine section must handle a 64-switch and thus requires at least 128 coefficients to accommodate this. For this reason, a 128-coefficient memory was integrated into this section.

3.3.4 Exhaust Section: Final Transformation

The exhaust section finalises the transformation by completing stage eight and performing scaling operations for INTT operations. It includes two sets of 128-coefficient memory, which supply data to a column of 4 butterfly units via a multiplexer. The multiplexer ensures that the correct coefficients are processed from memory. The butterfly units receive a new set of eight coefficients every cycle and output the results to the scaling unit.

The scaling operation is only required for INTT operations and is handled by a scaling unit that performs a modular multiplication to the output coefficients. This unit executes line 17 of Algorithm 2, ensuring that the transformed data is correctly formatted for use in Kyber and other PQC applications.

This section operates independently of any hardware in the other sections, allowing the starter and engine sections to begin processing a new transformation while the final stage is still executing. This overlapping execution improves the overall throughput of the design. To avoid potential data conflicts, this section uses two separate 128-coefficient memory blocks rather than sharing memory with the engine section. While a shared memory approach could reduce area usage, it would harm potential performance as the starter and engine sections would need to overwrite data still needed for the final stage. The additional memory cost can therefore be justified by the performance gain from the parallel execution.

3.3.5 Control Unit

The control unit coordinates all operations within the accelerator. It manages memory access, twiddle factor selection, synchronisation of different sections, external inputs such as the start signal and mode selection, and external outputs such as the result ready signal and the seven-segment display output. Once a start signal is received, the control unit locks in the selected mode (NTT or INTT) and ensures that all computations follow the appropriate sequence. It also manages interactions with external hardware, including interface signals for buttons, switches, and status LEDs on the FPGA.

Chapter 4

Implementation

4.1 Development Environment and Tools

The hardware design was implemented using SystemVerilog for the DE1-SoC FPGA board and was developed using the Intel Quartus Prime software suite. SystemVerilog was used due to both familiarity with the language and its ability to integrate with many other tools. The DE1-SoC board was chosen for its built-in Arm processor, its relatively low cost, and its wide availability. Quartus Prime has seamless integration with the DE1-SoC, making it a natural choice for working with this FPGA. Quartus also has robust timing and performance analysis tools and strong SystemVerilog support, helping improve the efficiency of the implementation and debugging process.

4.2 SystemVerilog Modules

The accelerator was implemented using a modular system where individual SystemVerilog hardware modules are integrated together to form larger components. Each hardware module fits into the overall hierarchy of the implementation, as shown in Figure 4.1, with the control unit's module acting as the top-level module. This modular approach greatly improved code reuse and allowed individual components to be more easily tested.

Parametrisation was implemented to utilise the modular design and allow compile-time configurability of certain elements of the accelerator. Some of the parameters

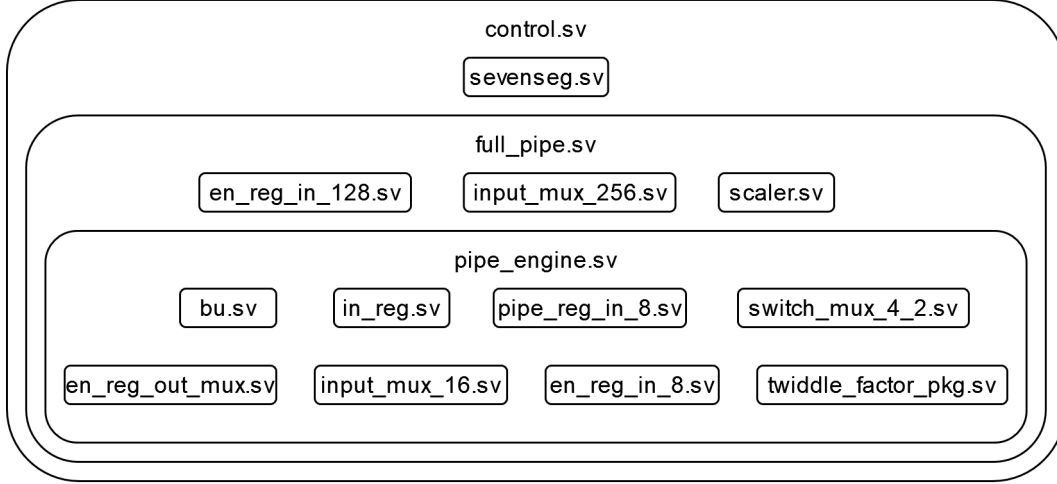


FIGURE 4.1: Diagram showing the SystemVerilog modules used to build up the implementation and the hierarchy of the modules.

used are the modulo value, scaling factor, bit-width, and twiddle factors. These parameters enable the accelerator to be adaptable for any 256-coefficient transformation scheme, as values are passed from higher-level modules to all smaller modules within the design. This flexibility ensures the hardware can be configured for different PQC applications without requiring significant and time-consuming modifications to the code.

4.3 Butterfly Unit

To improve code reuse and readability, SystemVerilog functions were implemented to handle the main operations performed by the butterfly units. These functions are modular adding, subtracting, multiplying, and modular reduction, as needed by Algorithms 1 and 2. Modular reduction is used in each of the modular arithmetic functions, making it vital to implement correctly and efficiently.

Therefore, this butterfly unit design implements the efficient Barrett reduction algorithm, shown in Algorithm 3, in the modular reduction function. The function implements the required precomputed value, m , for the Barrett algorithm as a local parameter in the SystemVerilog module. This means the Barrett algorithm can be easily adjusted for other modulo values. The implementation of this algorithm allows Quartus to synthesise much more efficient hardware for modular reduction compared to the hardware that would be synthesised with the normal modulo operator.

4.4 Memory Architecture

Pipeline registers in the design are implemented using simple flip-flops that are synthesised into hardware by Quartus. The engine memory is also built with flip-flops but has added enable signals that are managed by the control unit. Each of these enable signals control a set of memory containing eight coefficients, meaning the engine memory is built with 32 sets of eight-coefficient memory. This allows the control unit to precisely control where a set of eight coefficients from the butterfly units will be written within the engine memory. The added level of control is required to avoid data being overwritten when it is still needed for further processing. Figure 4.2 illustrates the memory access implementation and why control over which set of memory is enabled is particularly useful.

Similarly, the exhaust memory is also constructed with flip-flops and enable signals. However, it consists of 2 sets of 128-coefficient memory units instead of 32 sets of 8-coefficient memory units like the engine memory. Both sets of 128 have their own enable signal. This allows the entire engine memory to be written to either of the two memory sets when enabled. Adding more enable signals to this memory would be a waste of resources as more precise control over memory placement is not required in this section.

Additionally, a set of Read-Only Memory (ROM) was implemented to provide input data to the accelerator. The ROM has an address signal input that is controlled by the control unit to determine which set of coefficients to output. This ROM supplies the initial coefficients to the first pipeline register of the starter section, acting in place of external hardware, such as a CPU, that would be sending coefficients to be transformed. Although other input methods could have been used, such as switches on the FPGA, the use of this ROM greatly improved the testability of the system as inputs could be reliably repeated and easily changed in the code.

4.5 Control Unit

As the main coordinator in the design, the control unit was the most complex to implement. It contains four state machines, each responsible for managing different aspects of the system. The main state machine, shown in Figure 4.3, controls the overall operation by coordinating the other three through hardware

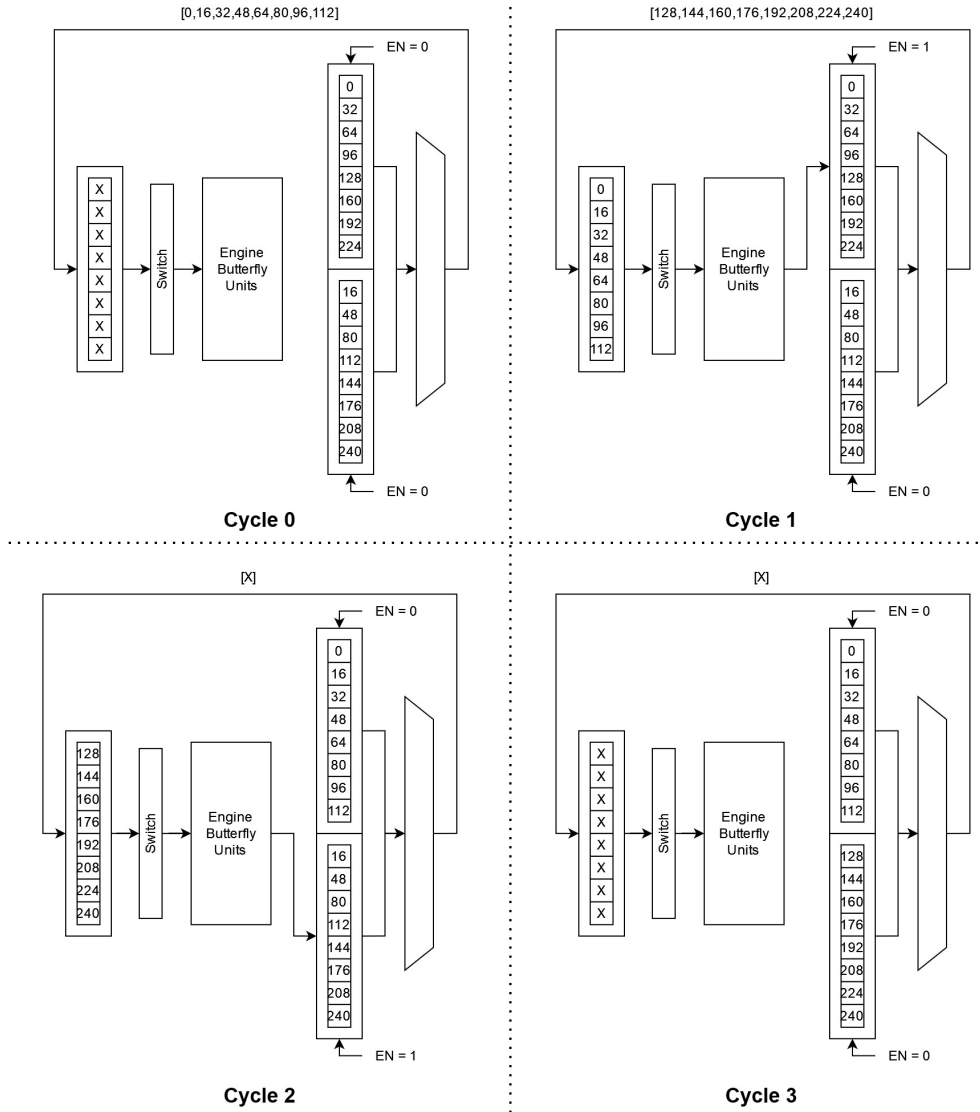


FIGURE 4.2: Diagram showing an example of the cycle-by-cycle flow of data to and from the engine memory in the engine section. Only 2 of the 32 sets of 8-coefficient memory are shown for simplicity, with all X symbols representing data irrelevant to this example.

signals. This main state machine determines when an operation begins, when the engine section is active, when the exhaust section runs, and when the entire operation is complete.

The remaining three state machines handle more specific tasks. One is responsible for assigning twiddle factors in the starter section, another manages twiddle factor assignments within the engine section, and the third controls memory read and write operations for the engine memory. Each of these state machines uses incrementing counters to generate control signals and determine when to transition

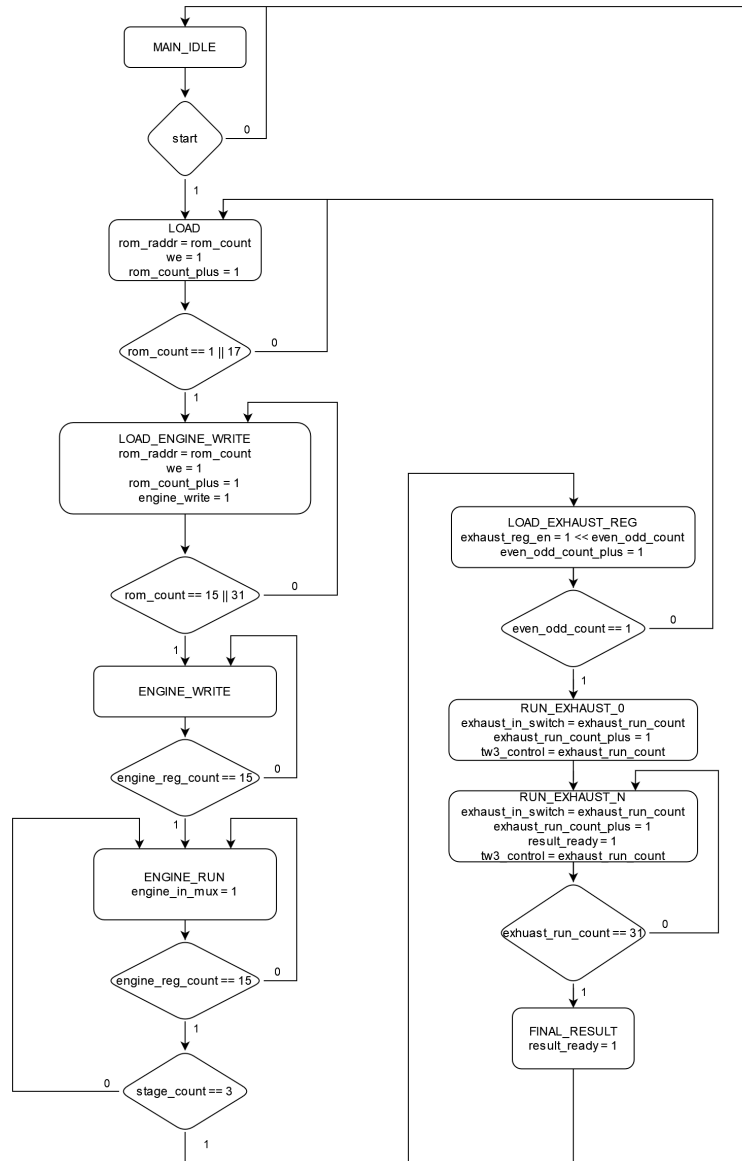


FIGURE 4.3: Flowchart showing the control unit's main state machine and the progression through its states.

between states. The flowcharts for every state machine can be found in Appendix B.

Although these state machines could have been consolidated into a single unified controller, separating them greatly simplified their implementation. The more modular approach made each state assignment flowchart easier to understand and sped up the overall development process.

4.6 Memory Access Patterns

As mentioned, one of the state machines controls the memory reads and writes for the engine memory. This state machine achieves this by controlling the signals that enable writing to each of the sets of eight-coefficient memory. The control of these enable signals allows the complex access patterns required to not overwrite data that still needs to be processed. Some of this complexity has already been shown in Figure 4.2. The root of this complexity is that the butterfly units require data from two different sets of eight-coefficient memory each cycle and thus only these two sets of memory can be safely overwritten.

Unfortunately, the complexity of these access patterns is exacerbated by the fact that the pattern by which two sets of memory are being accessed is different for each stage. The memory access pattern is therefore different for each stage of the processing in the engine section and the state machine has to accommodate these differences. This was implemented by adding separate flows in the state machine depending on what stage of processing was taking place. The additional flows resulted in a more complex state assignment chart and more resource usage but were necessary to ensure memory integrity.

4.7 Twiddle Factor Management

Each butterfly unit needs a specific twiddle factor for every single set of inputs it receives. For a 256-coefficient hardware accelerator, this involves handling potentially 1024 twiddle factors. Fortunately, many of these 1024 twiddle factors are duplicates and the number of unique twiddle factors is only 256. This implementation exploits this by only storing the 256 unique twiddle factors, using an index system to assign each butterfly unit with its correct twiddle factor from this list. While this does mean 1024 indices must be stored, the number of bits, and thus resources, required to store these indices is far fewer than storing 1024 whole twiddle factor values as each index is only 8 bits.

The twiddle factors are assigned to the butterfly units with SystemVerilog switch statements where the value of the control unit's control signals determine what index values apply to each unit, therefore determining which twiddle factor is assigned. However, the list of twiddle factors and the indices are stored as local parameters within the SystemVerilog module. This means that the twiddle factor assignments are synthesised into static physical hardware where multiplexers

simply route the correct twiddle factors depending on the control signal's value. Using this approach reduces the required computations of the accelerator as all the twiddle factor calculations and assignments are pre-computed. This is only possible because the set of twiddle factors for a specific PQC scheme configuration will not change during operation and can therefore be prepared at compile-time.

An auxiliary program was written in C to generate the twiddle factors and indices for any desired primitive root of unity and PQC scheme. This program replicates the NTT and INTT algorithms as described in algorithms 1 and 2 in software, outputting only the list of unique twiddle factor values and the list of corresponding indices. These lists can then be copied directly into the SystemVerilog code and are ready to be tested and verified.

Chapter 5

Testing and Verification

The primary goal of the accelerator is to produce accurately transformed final results. To achieve this, a systematic approach was adopted for testing and debugging during development. The testing strategy, shown in Figure 5.1, focussed on ensuring individual components were tested and verified before being integrated into larger components. This allowed components to be iteratively debugged and improved before being integrated into a larger system where the source of errors might become more elusive.

All simulation testing was conducted using ModelSim, a hardware description language simulator, that enables cycle-by-cycle verification of signal waveforms. Additionally, Quartus' internal simulation tools enabled further verification and ensured the hardware could be synthesised correctly. Initial testing was conducted using the Kyber version 1 modulo value of 7681 and a 16 bit width, with verification of alternative parameters conducted in further testing. Auxiliary software programs were developed and integrated into the testing strategy to allow efficient verification and debugging of the full system.

5.1 Stage-by-Stage Calculator

To ensure the accelerator produces accurate results, it's essential to know the final expected output values. A Python program was developed as a reference model to generate these expected results by implementing the accelerator's transformation algorithm in software. However, having only the final results is insufficient for

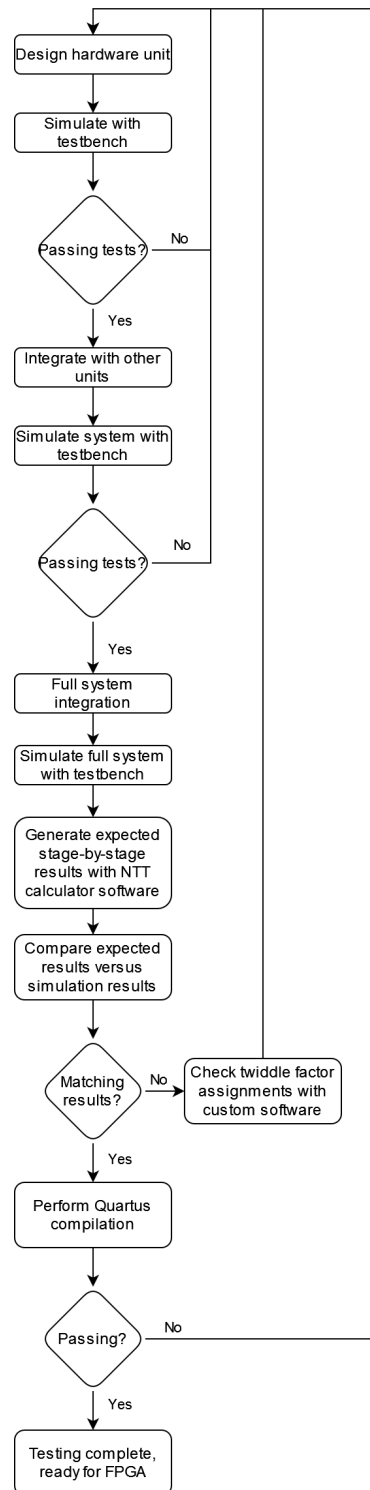


FIGURE 5.1: Flowchart showing the testing strategy used in testing and verifying the hardware accelerator at various stages of development.

robust testing and debugging as errors can occur at any stage of the transformation. To address this, the program was improved to also calculate stage-by-stage results for both NTT and INTT operations. This feature allows for stage-by-stage

verification of the hardware implementation, a feature not easily found in other NTT calculators. This approach significantly reduced debugging time as errors could be found in specific stages. Stage-by-stage verification also confirms the accuracy of the algorithm implementation and not just the final results. The program's accuracy was validated using known correctly transformed NTT and INTT operations.

Additionally, this software program is highly configurable, allowing results to be generated for any number of coefficients and choice of primitive root of unity. The program also checks that the chosen primitive root of unity is suitable, generates the inverse primitive root of unity for the INTT operation, and provides the required scaling factor needed for scaling the INTT operation output. The configurability and additional features of this program not only made full system testing more efficient and robust, but it also allowed comprehensive testing of certain sections of the design that produce results other than the final stage.

5.2 Component Testing

5.2.1 Butterfly Unit

The butterfly unit was the first component of hardware to be implemented and was therefore the first to be tested. As the backbone of the entire design, verifying the correctness of the butterfly unit's outputs was vital before any further progression with the implementation. Therefore, a testbench was created that instantiates two butterfly units in parallel, provides example input signals, and then outputs the transformed results.

Two butterfly units were instantiated as this allowed testing of 4-coefficient polynomials, a polynomial size that was easy to verify and had known correct examples in research [3]. The testbench verifies NTT and INTT operations by performing both operations. This is done by transforming and then reversing the transformation of the same data inputs. If the units are working correctly, the INTT operation should result in a scaled version of the original input to the NTT operation, as scaling is not performed in the butterfly units.

Figure 5.2 shows an example of the ModelSim simulation waveforms of the butterfly units correctly transforming $[1,2,3,4]$ into $[1467,2807,3471,7621]$ and back into $[4,8,12,16]$, using a primitive root of unity of 1925. The output scaled by 4 results

in the original input, indicating a correctly functioning set of butterfly units for both NTT and INTT operations.

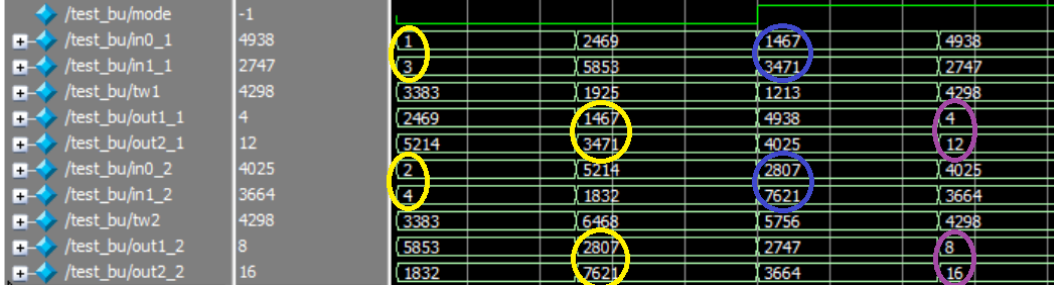


FIGURE 5.2: Annotated ModelSim simulation waveforms for a 4-coefficient NTT and subsequent INTT operation on an input of [1,2,3,4] and a primitive root of unity value of 1925.

In addition to the butterfly unit's testbench, another Python program was developed to further verify the operation of the butterfly unit with specific inputs. The primary purpose of this program is to identify errors in twiddle factor assignments. It takes the current inputs, current outputs, and the expected outputs as inputs and then determines the current twiddle factor being used and the correct twiddle factor for the unit. This functionality is particularly useful as a follow-up step after identifying a faulty output, as many development errors originated from incorrect twiddle factors.

5.2.2 Starter and Engine Sections

As the starter and engine sections share hardware, these sections were implemented and tested together. The initial testing of these sections was done using a 32-coefficient input example with a primitive root of unity of 330. This number of coefficients is ideal as it requires only five stages of processing, meaning that the first three are done in the starter section, one is done in the engine section, and the final one would be done in the exhaust section. Therefore the starter and engine sections can be tested without introducing unnecessary complexity that comes with larger numbers of coefficients. As with the butterfly unit, this testing involved verifying both NTT and INTT operations.

To verify the behaviour of the hardware in the testbench, the stage-by-stage calculator software was configured to the test parameters and the expected outputs for each NTT and INTT stage were calculated. The hardware's outputs for stages one,

two, three, and four all matched the expected results of the software model, indicating functioning starter and engine section implementations. This meant that these sections were ready to be integrated and tested with the exhaust section.

5.2.3 Starter, Engine, and Exhaust Sections

Testing the combined starter, engine, and exhaust sections uses the same testing parameters as the testbench for just the starter and engine sections. Using a 32-coefficient example is the simplest example where every section is used and can therefore be verified. With the addition of the exhaust section, final transformation results will now be produced and can therefore be matched against expected final results.

The stage-by-stage calculator was once again used and configured to the necessary parameters. In this case, every single stage of the accelerator could be verified and confirmed to be working correctly. Once testing of this section showed every stage to be functioning correctly, full system tests could take place.

5.3 Full System Testing

To verify Kyber compatibility, the full hardware accelerator system was tested using a 256-coefficient input, ensuring that all eight stages of transformation were correctly processed. The integration of the control unit significantly simplified the testbench design for this scenario, requiring only the start and mode signals to be manually controlled. As in the component testing, full system testing included both NTT and INTT verification.

Initial testing was performed using sequential values from 1 to 256 as the input coefficients and a primitive root of unity of 62. These inputs underwent an NTT operation and the subsequent outputs were fed into an INTT operation to validate both operation modes of the full system. The stage-by-stage calculator was configured with the specific parameters of this test case and was used to verify all eight stages of both the forward and inverse transformations. After confirming that the system produced accurate results for this initial scenario, additional tests were carried out to ensure the accelerator's correctness across a broader range of inputs.

As shown in Table 5.1, coefficient inputs were varied to ensure that the accelerator functioned correctly with many different inputs. This successful testing provided a good level of confidence in the accelerator’s ability to support the Kyber scheme. However, it was also important to verify that the parametrisation of the accelerator was functioning correctly, as all testing thus far was performed with only the Kyber scheme’s parameters.

TABLE 5.1: A table showing each of the coefficient inputs tested, whether each of their stages matched the software reference model, and whether the INTT of the NTT operation resulted in the original input.

Input	Software Model Match	INTT Output = NTT Input
1 to 256	Yes	Yes
2 to 257	Yes	Yes
101 to 356	Yes	Yes
1001 to 1256	Yes	Yes

5.3.1 Parameter Testing

The parametrisation of the design should allow the accelerator to be configured to any 256-coefficient transformation. Testing with different parameters was essential to verify that the parametrisation of the accelerator was implemented correctly. This testing was conducted by changing testbenches to use new parameter values and verifying that the outputs match the expected outputs from the stage-by-stage calculator. Table 5.2 lists some of the parameter values that were tested, each with a coefficient input of 1 to 256.

TABLE 5.2: A table showing the parameter values tested, whether each of their stages matched the software reference model, and whether the INTT of the NTT operation resulted in the original input.

Modulo	Root of Unity	Software Model Match	INTT Output = NTT Input
7681	1115	Yes	Yes
12289	113	Yes	Yes
12289	11854	Yes	Yes

These tests show that the parametrisation of the accelerator was successful and it can be configured to work with many other potential schemes that use 256 coefficients. Unfortunately, due to time constraints, one parameter that was not tested was the bit-width of the accelerator. Nevertheless, the successful simulation and testing of the full system meant that an attempt could be made to synthesise the design onto a physical FPGA board.

5.4 Quartus and FPGA Testing

After successfully simulating the design in ModelSim and confirming that the hardware functioned correctly, the next step was to synthesise the design using Quartus. During this process, Quartus conducted its internal simulations, as shown in Figure 5.3, to verify the hardware functionality and ensure it could be implemented on the FPGA.

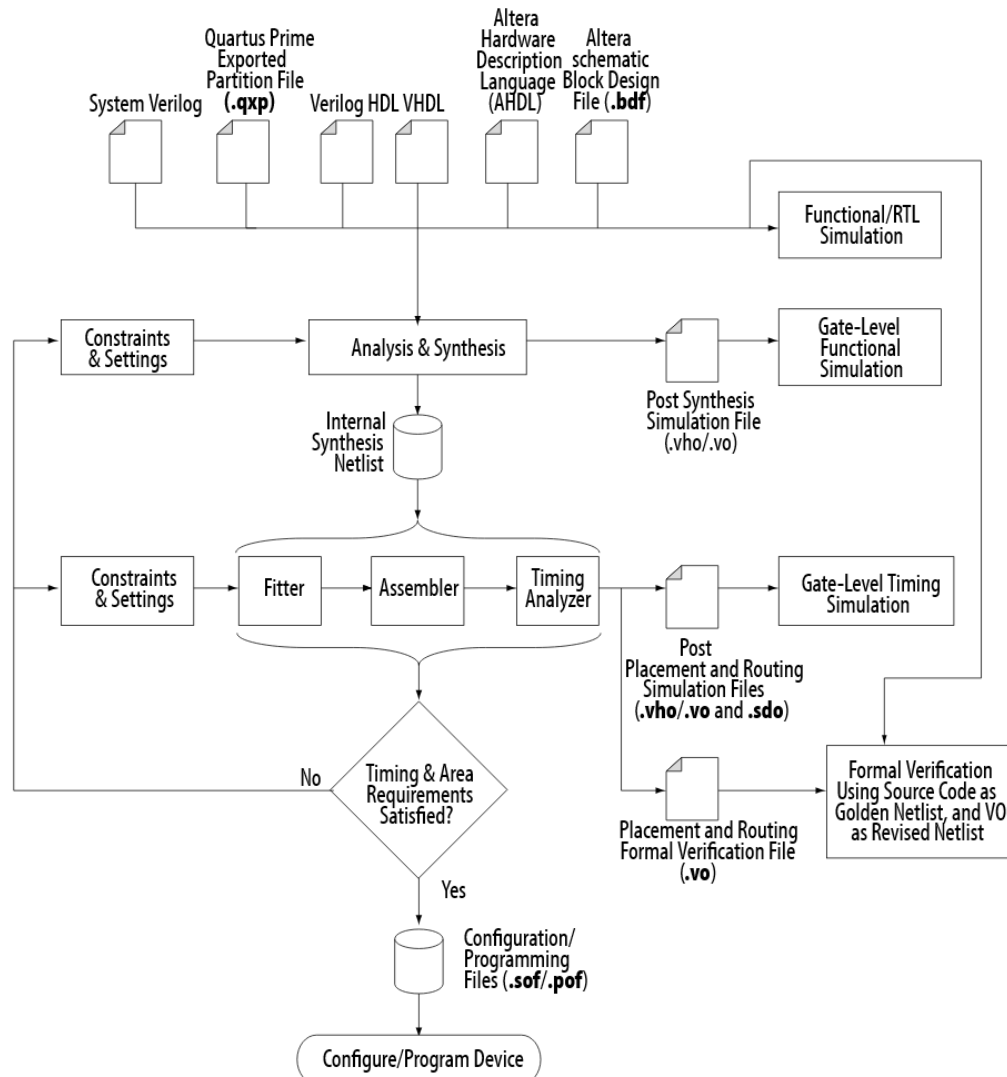


FIGURE 5.3: Diagram showing Quartus' design and compilation flow, including all the simulations performed by Quartus during compilation. Taken from [12]

Once synthesis was complete, the design was programmed onto the FPGA. The buttons and switches on the FPGA served as inputs to the accelerator and the

seven-segment displays showed the accelerator's outputs in hexadecimal form, using edited code provided by Professor Mark Zwolinski. The accelerator's functionality was then verified by comparing these hexadecimal outputs with the expected results.

Additionally, Quartus provides valuable insights into the design's resource utilisation and timing characteristics, making it easier to evaluate and compare the implementation.

Chapter 6

Evaluation

6.1 FPGA Performance

Once the design’s outputs were verified to be correct, the performance characteristics of the design were captured using the built-in analysis tools of the Quartus software suite. As seen in Table 6.1, the design was synthesised for three different FPGAs: the DE1-SoC’s Cyclone V FPGA, the Cyclone 10 GX FPGA family, and the more powerful Stratix 10 GX family. Each of these were synthesised with Quartus’ balanced performance setting. The Cyclone 10 GX and Stratix 10 GX families later provide a more fair comparison between this design and the other FPGA implementations that use Artix-7 and Virtex-7 FPGAs [13]. All performance evaluations of the accelerator implementation were conducted with the Kyber scheme modulo value of 7681.

TABLE 6.1: A table showing the three different FPGA families that the accelerator was synthesised for, illustrating the differences in resource usage and maximum clock frequency. ALM, FF, DSP, and BRAM stand for Adaptive Logic Modules, Flip-Flops, Digital Signal Processors, Block RAMs respectively.

Platform	ALM/FF/DSP/BRAM	Frequency (MHz)	NTT/INTT Cycles
Cyclone V	13,052/4651/87/0	13.71	208
Cyclone 10 GX	7,538/4641/139/0	26.38	208
Stratix 10 GX	7,829/4071/139/0	35.58	208

6.2 Performance Comparisons

6.2.1 Arm Cortex-M3

The main purpose of hardware accelerators is to execute specific operations faster and more efficiently than general hardware. The Arm Cortex-M3 is a suitable example of a general CPU core that might benefit from a hardware accelerator, such as the one implemented in this project. As shown in Table 6.2, this Arm core achieves an NTT latency that is orders of magnitude worse than the accelerator's latency. The accelerator achieves a result up to 116 times faster than the Arm core, showing the effectiveness of this accelerator in speeding up CPU cores.

TABLE 6.2: A table comparing the clock frequency, number of NTT operation clock cycles, and the calculated NTT latency of an Arm Cortex-M3 core against the project's accelerator implementation on three FPGAs.

Platform	Frequency (MHz)	NTT Cycles	Latency (μ s)
Arm Cortex-M3 [14]	16	10,819	676.188
Cyclone V	13.71	208	15.171
Cyclone 10 GX	26.38	208	7.885
Stratix 10 GX	35.58	208	5.846

Figure 6.1 better illustrates the dramatic speed improvement that can be achieved by implementing a hardware accelerator that is designed for a specific operation. However, this hybrid accelerator design is not the only FPGA NTT hardware accelerator that exists and thus it must be evaluated against other implementations.

6.2.2 Other FPGA Accelerator Implementations

As previously mentioned, this accelerator design must ideally be superior in at least one performance metric compared to other implementations to justify its use in a certain application scenario. The performance metrics assessed for this accelerator are latency, throughput, and area usage. These metrics were chosen due to the wide availability of data on these metrics for other implementations, allowing comparisons to be easily made.

6.2.2.1 Latency

This metric measures the total time taken before an NTT or INTT operation is completed. Table 6.3 shows the NTT latency of this project's accelerator compared

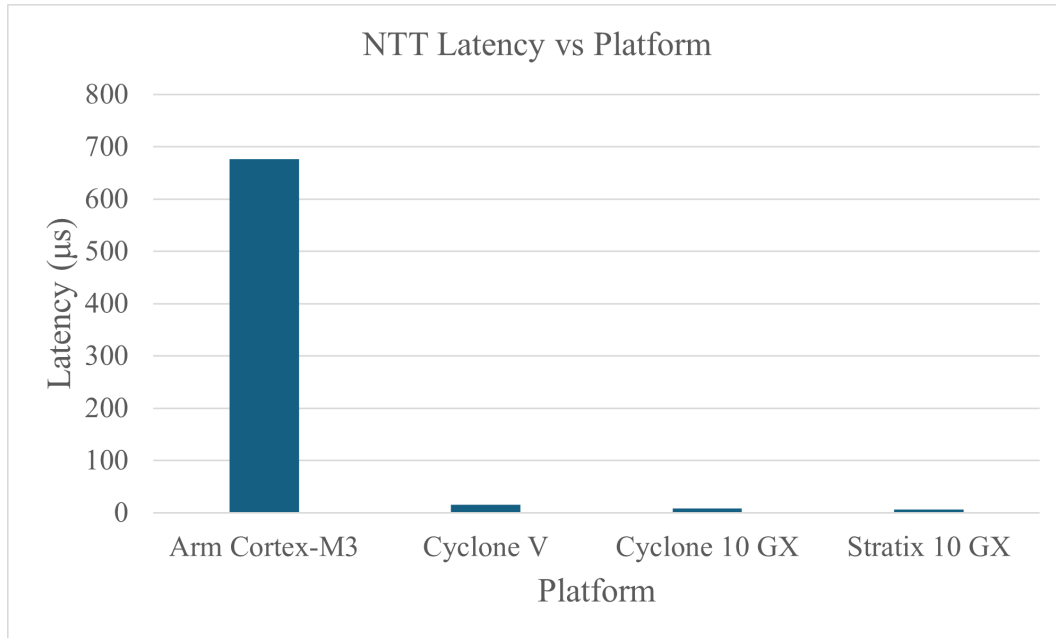


FIGURE 6.1: A bar graph diagram showing the NTT latency of the Arm Cortex-M3 against the project’s accelerator implementation on three different FPGAs.

to some other 256-coefficient accelerators. These results show that the latency of this accelerator is comparable to other published accelerators. However, there are areas for improvement that become particularly noticeable from this table.

TABLE 6.3: A table comparing the maximum clock frequency, number of NTT operation clock cycles, and calculated NTT latency of this project’s accelerator design against other comparable FPGA accelerator implementations.

Platform	Frequency (MHz)	NTT Cycles	Latency (μs)
Cyclone 10 GX	26.38	208	7.885
Artix-7 [15]	190	904	4.758
Stratix 10 GX	35.58	208	5.846
Virtex-7 [11]	174	1052	6.046
Virtex-7 [11]	186	156	0.839
Virtex-7 [11]	167	95	0.569

Although the number of clock cycles for this accelerator is lower than some of the others, the maximum clock frequency is significantly lagging behind the other designs. The main cause of this is likely long combinational logic paths between clocked registers. These long paths restrict the maximum frequency as the whole system has to wait for data to pass through the path before a clock signal can be sent.

Some of the long paths are likely caused by the choice of implementing the design with many very large switch statements which are synthesised into many successive

multiplexers. Each multiplexer adds additional delay to the path and these quickly add up to very large delays in the logic path, restricting clock frequency and the overall performance of the accelerator.

6.2.2.2 Throughput

The throughput of these accelerators can be considered as the time taken to complete several operations. This is a useful metric for systems that need to perform many NTT or INTT operations one after another. PQC schemes will often need to do at least two consecutive NTT operations as the two polynomials being multiplied together both need to be transformed.

Table 6.4 shows the theoretical throughputs of each of the implementations when considering the time taken to perform ten NTT operations sequentially. From this table, the positive effect of the accelerator's unique design can be seen where the average NTT latency for the ten operations is decreased by 14.7% compared to a single operation shown in Table 6.3. This average latency will decrease further with larger numbers of subsequent NTT operations. This is because the number of clock cycles required for each subsequent operation decreases from 208 to 174 after the first one.

TABLE 6.4: A table comparing the throughput of this project's accelerator against other FPGA implementations using the latency of ten NTT operations and the average NTT latency of these operations.

Platform	10 NTT Latency (μ s)	Average NTT Latency (μ s)
Cyclone 10 GX	67.25	6.725
Artix-7 [15]	47.58	4.758
Stratix 10 GX	49.86	4.986
Virtex-7 [11]	60.46	6.046
Virtex-7 [11]	8.39	0.839
Virtex-7 [11]	5.69	0.569

Unfortunately, the throughput is somewhat dependent on the single operation latency of the accelerator and, as already discussed, the latency of this accelerator has room for improvement compared to the other implementations. To overcome this poor single-operation latency, the accelerator would need to reduce the latency of the subsequent operations to below the single-operation latency of all the other implementations. This would mean that eventually, at a certain number of NTT operations, this design would prevail in throughput performance. However, this is

not the case and, with the current implementation, the design is not superior in terms of throughput.

6.2.2.3 Area Usage

Area usage in the context of FPGAs corresponds to the resources used in synthesising the design onto the FPGA. This metric is particularly important when considering implementing accelerators in resource-constrained environments such as embedded systems. Table 6.5 shows the resource and area usage of each of the implementations assessed in this project.

TABLE 6.5: A table comparing this project’s accelerator implementation’s resource utilisation against other FPGA implementations. **ALMs converted to Look-Up Tables (LUTs) by increasing the value by 33% to roughly convert Intel’s eight input LUT to AMD’s six input LUT [16][17].*

Platform	LUTs	FFs	DSPs	BRAMs
Cyclone 10 GX	10,026*	4641	139	0
Artix-7 [15]	948	352	1	2.5
Stratix 10 GX	10,413*	4071	139	0
Virtex-7 [11]	2,128	1144	8	3
Virtex-7 [11]	10,973	5422	64	12
Virtex-7 [11]	61,731	17,846	256	48

As seen in this table, the project’s resource and area usage is comparable to some other implementations but is not superior. One key difference seen in this table is the lack of BRAM utilisation with this project’s implementation, indicating a potential improvement in future versions of this design. Although the area usage is similar to some of the other implementations, this area usage can only be properly evaluated in a more holistic view of the implementation, taking into account other performance metrics.

6.2.2.4 Overall FPGA Comparison

When comparing FPGA implementations, it is important to assess and compare all of the performance metrics together. This is because one performance metric being superior might justify poorer performance in other metrics. Figure 6.2 displays each implementation’s performance in area usage, latency, and throughput.

Unfortunately, as shown in Figure 6.2, it is difficult to justify using this project’s implementation in its current form as other implementations outperform it on the

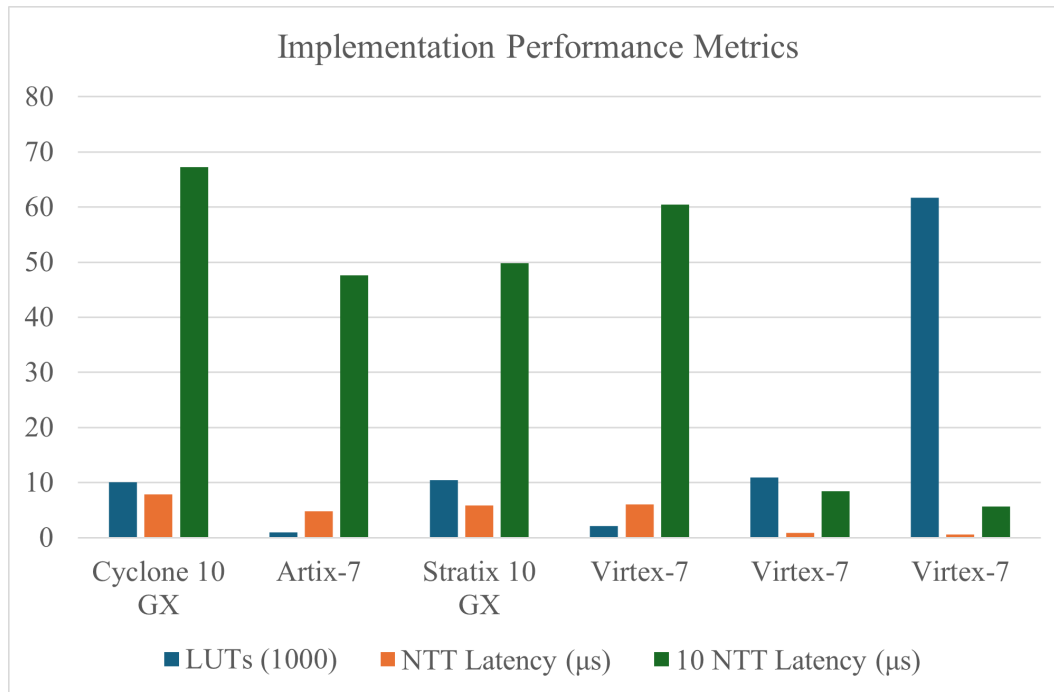


FIGURE 6.2: A bar graph showing each FPGA accelerator implementation’s number of LUTs used (in thousands), NTT latency, and 10 NTT operation latency to represent throughput.

assessed metrics. For example, the Artix-7 implementation uses fewer resources, has lower NTT latency, and better throughput for ten NTT operations.

However, given the limited time for optimisations, this project’s implementation is competitive with other implementations that have had considerably more time allocated to their development. Further optimisations and adjustments to this project’s implementation could improve performance metrics to the extent that it is justifiable for some application scenarios.

6.2.3 Performance Overview

This project’s accelerator design achieves typical performance of a hardware accelerator when assessing its performance against general hardware such as Arm cores. Evaluating the accelerator against other existing FPGA implementations shows that the, while the implementation is competitive in many areas, it is not superior in any performance metrics that were assessed. The specific effect of the unique design on the accelerator’s performance is difficult to discern due to an oversight in the project’s evaluation strategy.

An oversight was made in not producing a purely iterative implementation of this project's accelerator for evaluation purposes. Without this implementation, it becomes very difficult to fairly assess what effect the pipelined elements of the accelerator's design had on performance. Comparing the design to other iterative implementations provides some useful comparison but introduces too many unknown variables to easily identify the specific effect of the unique configuration. Nevertheless, good progress was made in developing this accelerator and achieving the overall goals of the project.

6.3 Project Progress

When assessing the project goals, the project was successful in meeting its main goal of building a working and configurable NTT hardware accelerator. This accelerator was implemented and verified with a rigid testing strategy, ensuring that this goal was met. Unfortunately, a decision had to be made to not focus on the goal of integrating the accelerator with a CPU on the FPGA. Working with the built-in Arm core on the FPGA turned out to be much more complicated and time-consuming than initially anticipated. Fortunately, data was already available on the performance of an Arm core when running NTT and INTT operations, allowing a comparison to be made. This meant that the goals related to the integration of the CPU core could be deprioritised without jeopardising the larger goal of the project.

Although the accelerator could be used for a full encryption and decryption test with a PQC scheme, the lack of a CPU to handle the operations not performed by the accelerator means that this could not be easily tested and verified. This means that the initially proposed stretch goal of this project is only partially achieved and would likely be achieved with more time.

6.4 Project Management

6.4.1 Time Management

Time was well managed throughout the project, despite some setbacks in progression. A Gantt chart was used as the main time management tool for the project and was updated as the specifics and timings of the project evolved. The versions

of this Gantt chart can be seen in Appendix A. A strong attempt was made to follow the initial version of the Gantt chart but unfortunately this turned out to be an ambitious timeline. Various setbacks and unforeseen delays resulted in the Gantt chart being slightly altered and certain tasks removed.

As mentioned previously, the integration of the Arm CPU on the FPGA with the accelerator was not achieved as other tasks were prioritised. Unfortunately, some time was wasted trying to get the CPU integration working before the decision was made to focus on other tasks. This meant that valuable time was lost that could have been better spent on other tasks.

However, the biggest unexpected delay was the development of the accelerator itself. A large amount of time was initially allocated to this task as it was the main goal of the project. However, this time was not nearly enough as the development took far longer than anticipated. This was caused by large complexities in the design, such as the complexity of the twiddle factor assignments and required memory accesses, which only became apparent during development. If these complexities had been better foreseen, the timing could have been more realistic. The delay in the accelerator development had a knock-on effect where later tasks could not be completed. One of these tasks was the optimisation of the design to improve performance.

Initially, time was allocated near the end of the project to perform optimisations on the design and explore their performance improvements. As seen from the performance evaluation section, the design could have benefitted greatly from optimisations and would probably have provided a fairer comparison of this design to other designs. Nevertheless, despite timing setbacks, significant progression was made and was helped by the management of risks throughout the project.

6.4.2 Risk Management

As seen in the risk evaluation table in Appendix A, there were many risks to manage in the project. Poor management of these risks could have led to a significant loss of time in the project and a poor final product. Fortunately, the contingency planning and risk mitigations developed early on in this project protected large amounts of time from being lost.

One risk that was identified was the CPU core being too difficult to use. This turned out to be true and the mitigation had to be deployed. If this risk and its

mitigation had not been identified, even more time would have been potentially wasted on this element of the project. This highlighted the importance of trying to evaluate risks early in the development of the project so that mitigations can be deployed quickly and effectively when needed.

The use of version control software such as Git and GitHub was mainly to mitigate any data loss risk by storing data online. However, this version control approach ended up contributing well to the overall time management of the project. The version control software acted as a mapped timeline of progression for the project with each commit acting as a checkpoint of the work achieved so far. This was also particularly helpful in allowing work to be undone by restoring to a previous version. Therefore, the use of this software helped mitigate the data loss risk and helped with time management in the project.

Fortunately, many of the identified risks in this project did not end up occurring and the project largely progressed without hindrance from external factors. Nevertheless, if any of these risks had occurred, the mitigations in place would have drastically diminished their negative effects.

Chapter 7

Conclusions

This report has presented the background, design, implementation, and evaluation of a custom hardware accelerator for NTT and INTT operations, an increasingly demanded set of operations as PQC schemes are adopted. The goal of this project was to address this demand by building a unique implementation of an NTT hardware accelerator and evaluating its performance.

This goal was ultimately achieved with a hybrid accelerator design, combining elements of traditional pipelined and iterative designs. This design met the defined specification and is a 256-coefficient, Kyber-compatible accelerator. A rigid testing and verification system confirmed the functioning of this accelerator before conducting performance evaluations.

The accelerator managed to produce a result for NTT operations up to 116 times faster than a stand-alone Arm Cortex-M3 core, illustrating its success as an accelerator. Unfortunately, this hybrid design did not outperform already existing FPGA accelerator implementations in terms of area usage, latency, or throughput. This makes it difficult to justify this design over others in its current form.

This design and implementation has significant room for optimisation and acts as a solid foundation for further development to take place. Exploring new accelerator designs is vital in meeting the demand for efficient PQC scheme processing in an increasingly wide range of environments.

7.1 Future Work

Despite the significant progress and success of the project, timing constraints have resulted in many avenues for future work and research to explore. Some areas for future work include:

- Optimising the current design to remove long path delays and improve maximum clock frequency, making the design more competitive against existing implementations.
- Integrating the accelerator with a CPU core and running a full PQC scheme test.
- Exploring different configurations of the hybrid design by changing the number of butterfly units used for each section.
- Instantiating multiple starter and engine sections of the design in parallel to explore the trade-offs of this change.
- Exploring the power consumption of this hybrid design and comparing it with other implementations.

Many of these tasks would provide clearer insights into the viability of this unique NTT hardware accelerator design and the future it may have in PQC processing.

Bibliography

- [1] R. Bavdekar, E. Jayant Chopde, A. Agrawal, A. Bhatia, and K. Tiwari, “Post quantum cryptography: A review of techniques, challenges and standardizations,” in *2023 International Conference on Information Networking (ICOIN)*, 2023, pp. 146–151.
- [2] P. K. Pradhan, S. Rakshit, and S. Datta, “Lattice based cryptography : Its applications, areas of interest future scope,” in *2019 3rd International Conference on Computing Methodologies and Communication (ICCMC)*, 2019, pp. 988–993.
- [3] A. Satriawan, I. Syafalni, R. Mareta, I. Anshori, W. Shalannanda, and A. Barra, “Conceptual review on number theoretic transform and comprehensive review on its implementations,” *IEEE Access*, vol. 11, pp. 70 288–70 316, 2023.
- [4] E. W. Weisstein, “Convolution,” <https://mathworld.wolfram.com/Convolution.html>.
- [5] Y. Wang, “Artificial-intelligence integrated circuits: Comparison of gpu, fpga and asic,” *Applied and Computational Engineering*, vol. 4, pp. 99–104, 06 2023.
- [6] L. Wan, F. Zheng, G. Fan, R. Wei, L. Gao, Y. Wang, J. Lin, and J. Dong, “A novel high-performance implementation of crystals-kyber with ai accelerator,” in *Computer Security – ESORICS 2022*, V. Atluri, R. Di Pietro, C. D. Jensen, and W. Meng, Eds. Cham: Springer Nature Switzerland, 2022, pp. 514–534.
- [7] W. Tan, A. Wang, Y. Lao, X. Zhang, and K. K. Parhi, “Pipelined high-throughput ntt architecture for lattice-based cryptography,” in *2021 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*, 2021, pp. 1–4.

- [8] D.-e.-S. Kundi, Y. Zhang, C. Wang, A. Khalid, M. O'Neill, and W. Liu, "Ultra high-speed polynomial multiplications for lattice-based cryptography on fpgas," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 4, pp. 1993–2005, 2022.
- [9] P. Barrett, "Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor," in *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, ser. Lecture Notes in Computer Science, vol. 263. Springer, 1986, pp. 311–323.
- [10] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "Crystals-kyber algorithm specifications and supporting documentation (version 3.01)," Tech. Rep., 2021.
- [11] K. Derya, A. C. Mert, E. Öztürk, and E. Savaş, "CoHA-NTT: A configurable hardware accelerator for NTT-based polynomial multiplication," Cryptology ePrint Archive, Paper 2021/1527, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1527>
- [12] "Quartus prime standard edition user guide — design compilation," Intel, Tech. Rep., 2021. [Online]. Available: <https://cdrdv2-public.intel.com/666615/ug-qps-compiler-683283-666615.pdf>
- [13] "An 307: Intel fpga design flow for amd xilinx users," Intel, Tech. Rep., 2024. [Online]. Available: <https://cdrdv2-public.intel.com/816592/an307-683562-816592.pdf>
- [14] D. O. C. Greconici, M. J. Kannwischer, and A. Sprenkels, "Compact dilithium implementations on cortex-m3 and cortex-m4," Cryptology ePrint Archive, Paper 2020/1278, 2020. [Online]. Available: <https://eprint.iacr.org/2020/1278>
- [15] F. Yaman, A. C. Mert, E. Öztürk, and E. Savaş, "A hardware accelerator for polynomial multiplication operation of crystals-kyber pqc scheme," in *2021 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2021, pp. 1020–1025.
- [16] "Cyclone 10 gx device overview," Intel, Tech. Rep., 2019. [Online]. Available: <https://cdrdv2-public.intel.com/670531/c10gx-51001-683485-670531.pdf>

-
- [17] “Stratix 10 gx/sx device overview,” Intel, Tech. Rep., 2024. [Online]. Available: <https://cdrdv2-public.intel.com/670537/s10-overview-683729-670537.pdf>

Appendix A - Project Management

This appendix includes all project management related information and images. The three presented Gantt charts are, in order, the initial chart produced at the start of the project, the updated chart for the progress report, and the final chart representing what was actually achieved. This appendix also contains the risk evaluation chart and the personal skills audit created at the beginning of the project.

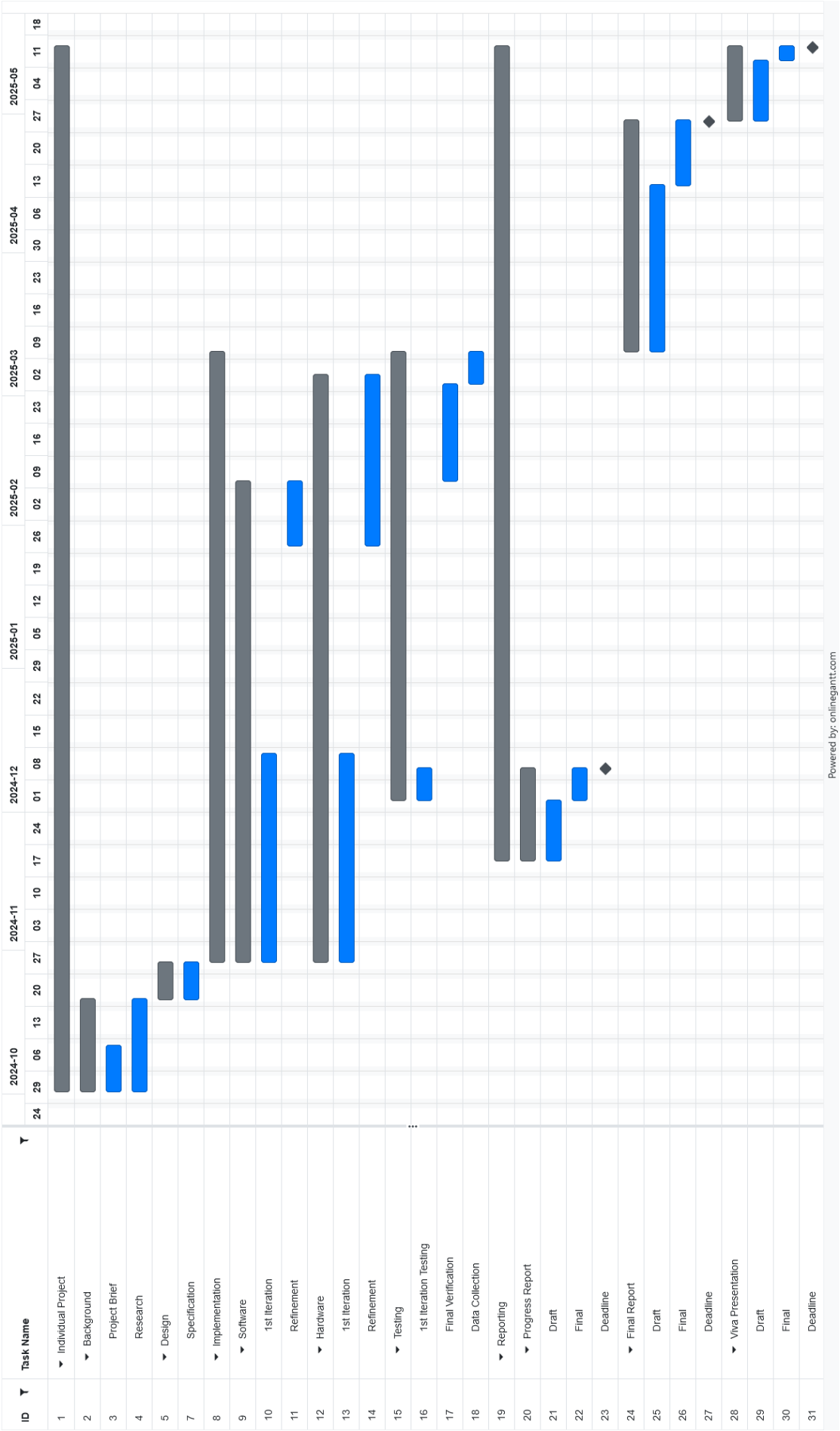
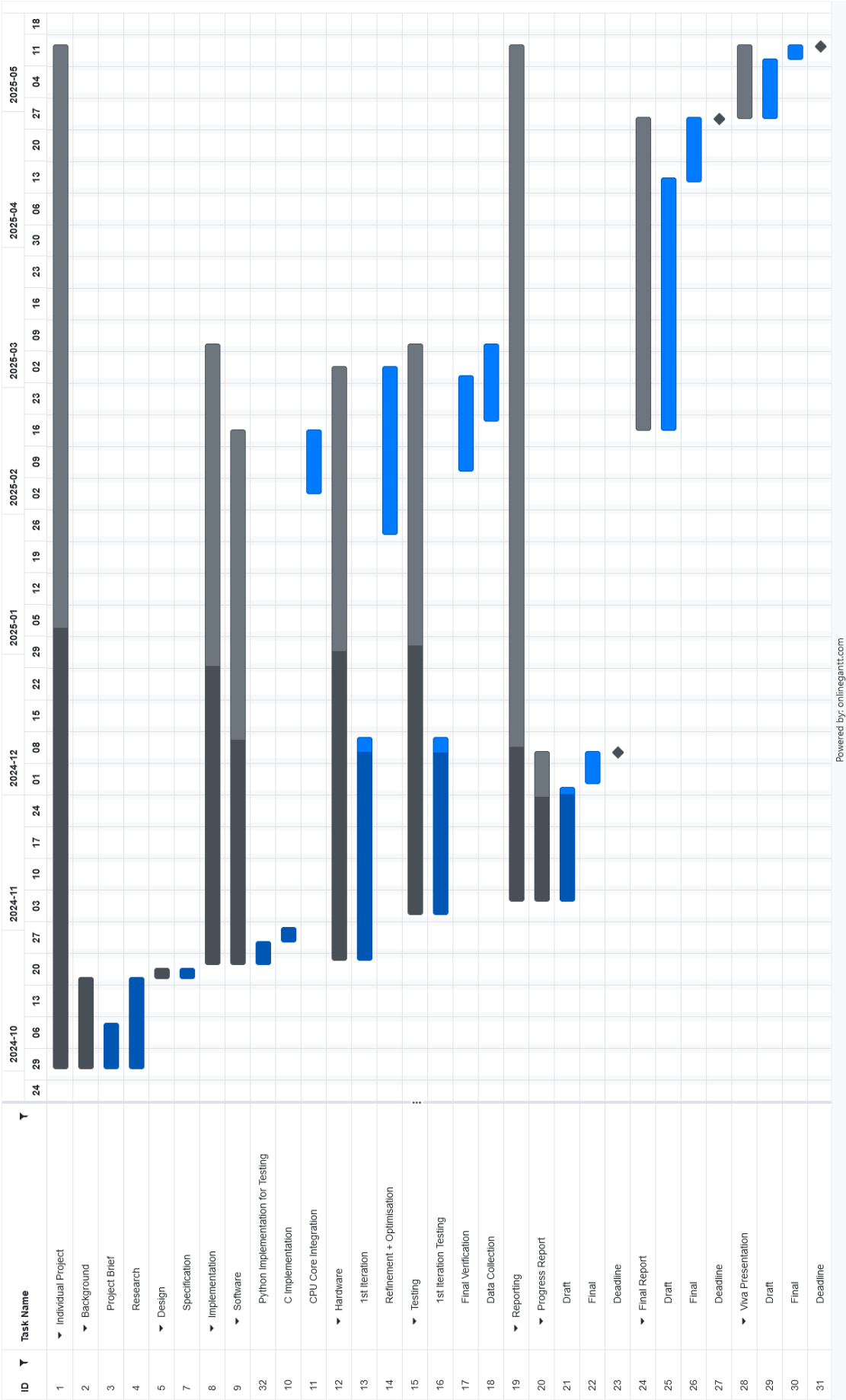


FIGURE 1: Gantt chart for project planning, as of 08/10/2024.



Powered by: onlinegantt.com

FIGURE 2: Updated Gantt chart for project planning, as of 01/12/2024.

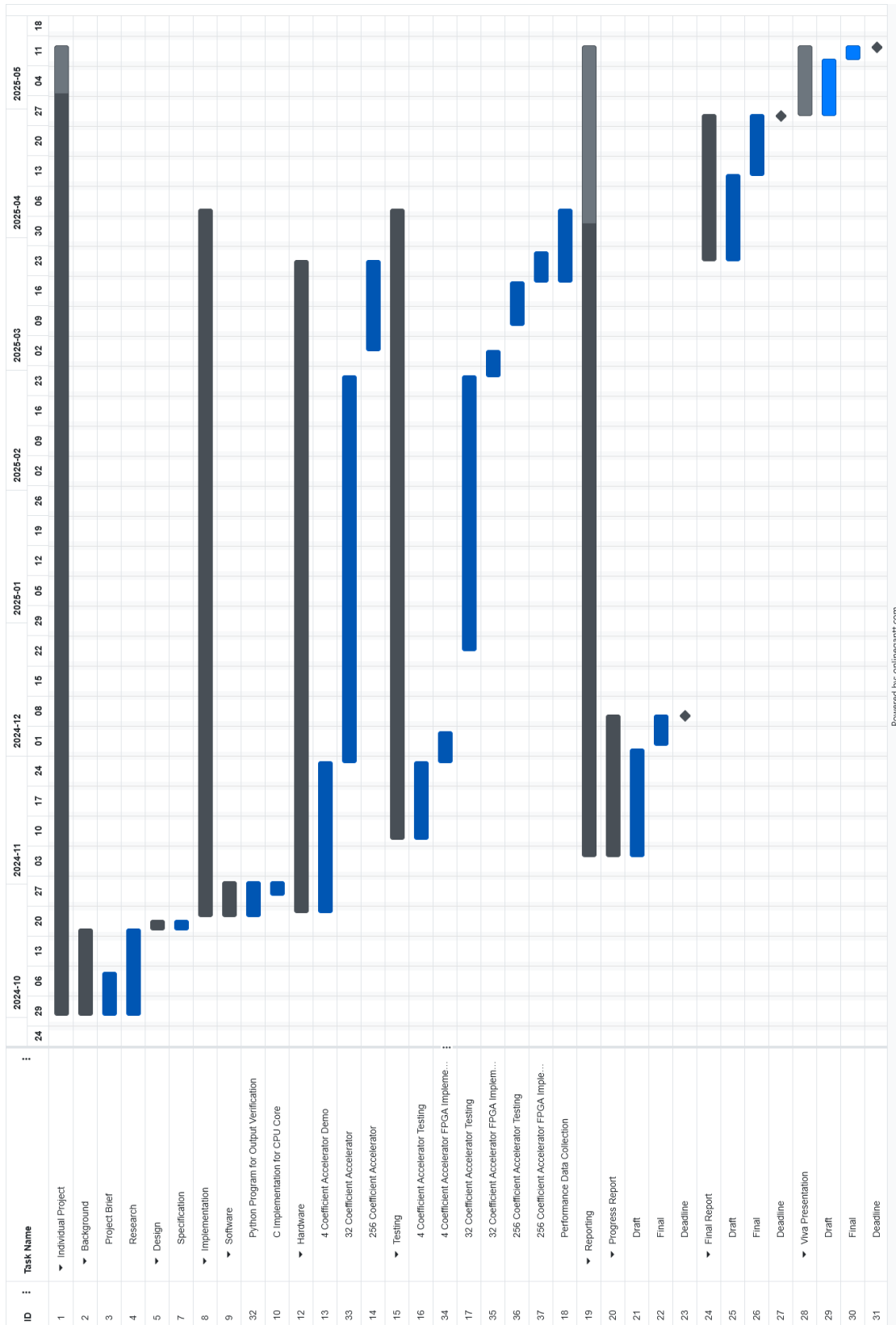


FIGURE 3: Final Gantt chart for the project, as of 29/04/2024.

Risk Evaluation Table:

Problem	Impact	Likelihood	Risk	Mitigation	Impact	Likelihood	Residual Risk
Portions or all of work is corrupted or lost.	5	2	10	All work is backed up to OneDrive. GitHub will be used for all code. Backups on external hard drive once a week.	5	0	0
CPU core on FPGA is too difficult to use.	4	3	12	Investigate viability of CPU core early in project. Can create additional hardware to fulfil role of CPU core if needed, using development board I/O to operate.	2	3	6
Project laptop is lost or broken.	4	2	8	All files kept online to access from university computers/loan laptop while laptop is being replaced.	1	2	2
FPGA kit is lost or broken.	4	2	8	FPGA will only be used at home or in B16 Lab and not moved unnecessarily. Replacement kits available from Lab 2.	1	2	2
Difficulty with FPGA operation or SystemVerilog	4	3	12	Help is available from university lecturers (Mark Zwolinski) and FPGA kit is used in labs so general help with them is available.	1	3	3

Impact and Likelihood scored from 0 (low) to 5 (high).

Risk = Impact $\times$ Likelihood, therefore a score of 0 (low) to 25 (high).

FIGURE 4: Risk evaluation table describing risks and mitigations to those risks for the project.

Skill	Proficiency (0-5)
Technical	
SystemVerilog	3
C Programming	5
Python Programming	4
Quartus/FPGA Programming	2
Other	
Writing	4
Research	3
Planning	4

FIGURE 5: Skills audit on areas of strength and potential weakness.

Appendix B - State Machine Charts

This appendix includes all the state assignment charts and flows taken by each of the four state machines used in the accelerator's implementation.

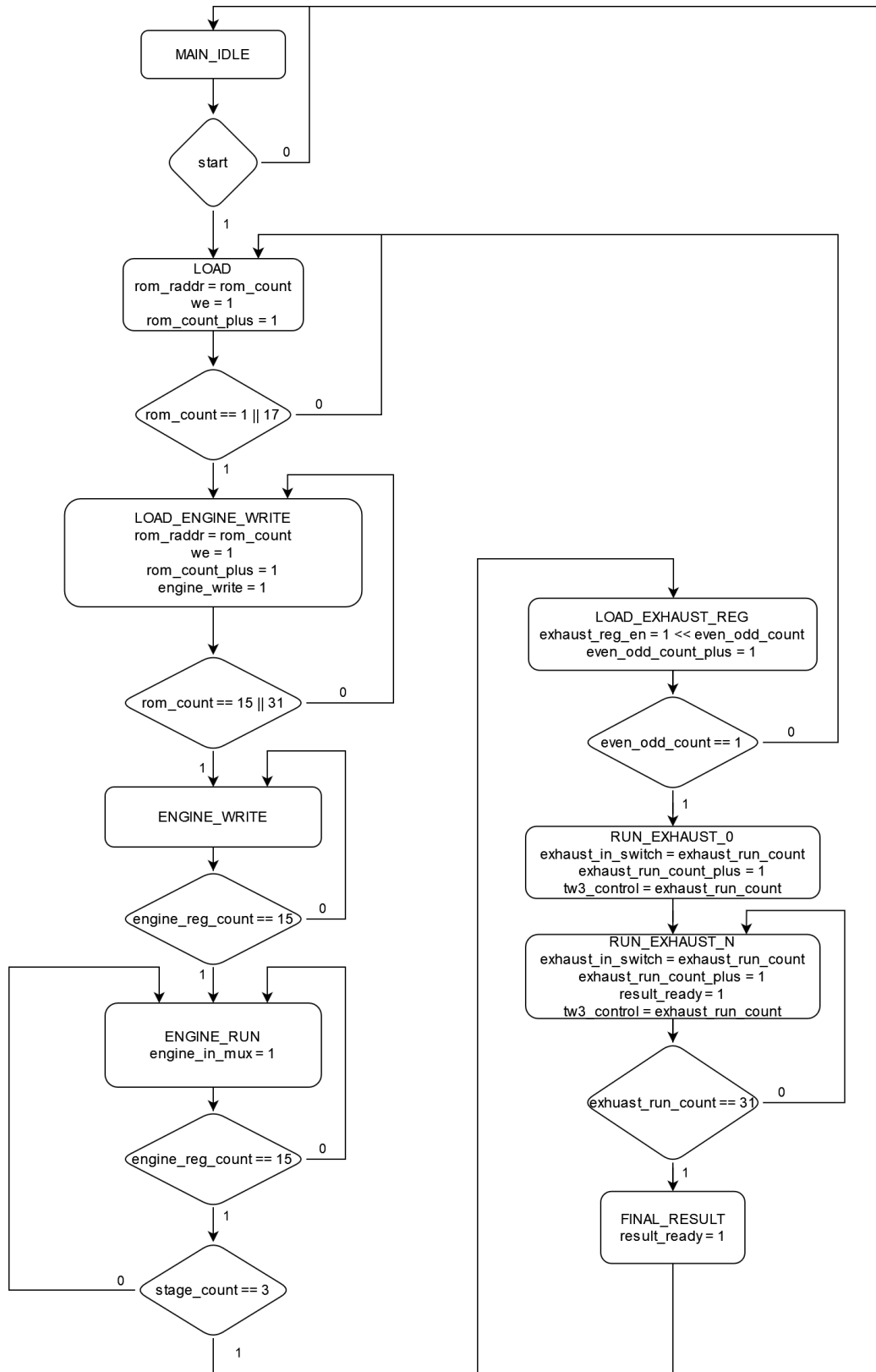


FIGURE 6: Flowchart showing the control unit's main state machine and the progression through its states.

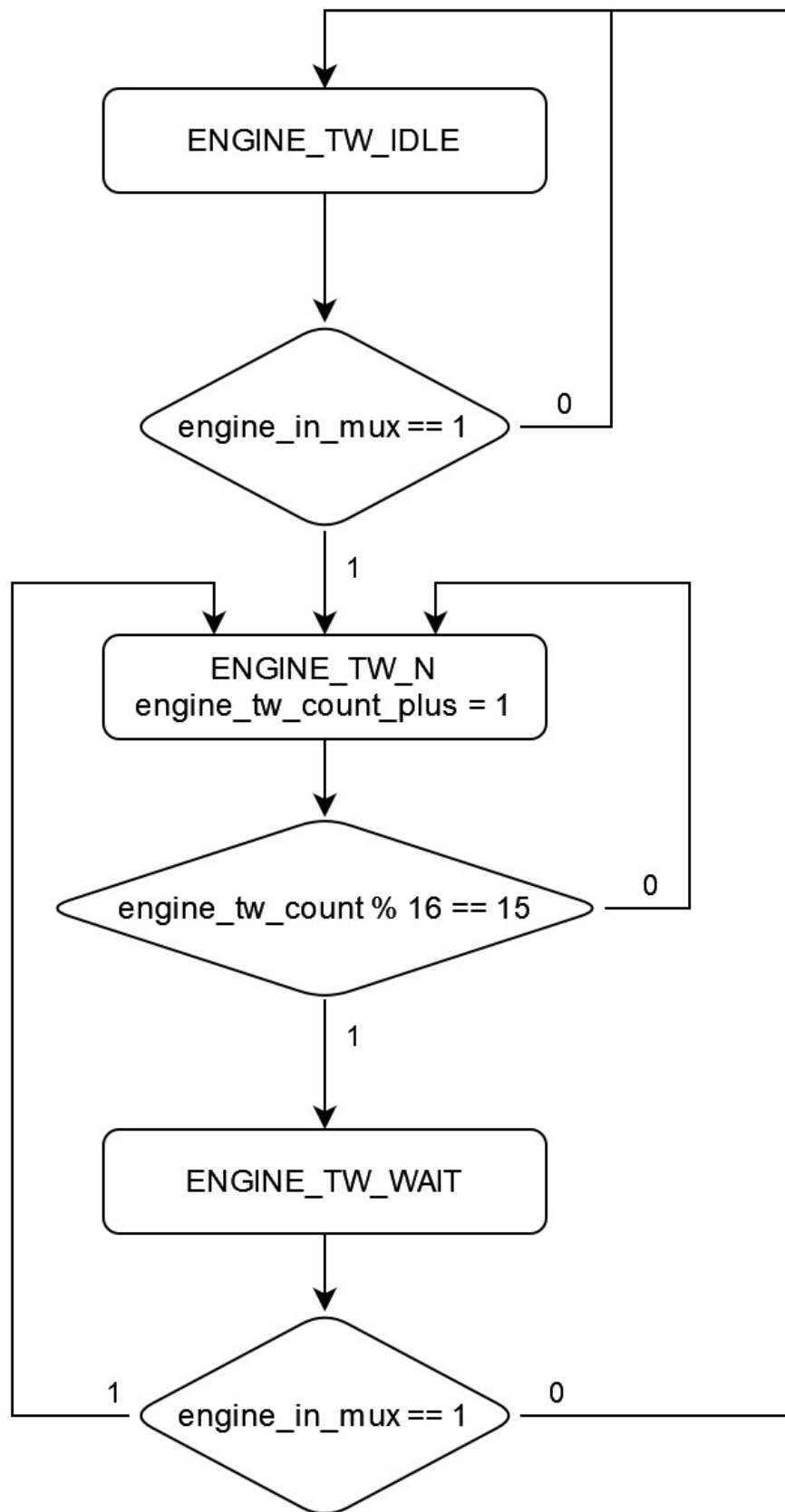


FIGURE 7: Flowchart showing the control unit's engine section twiddle factor state machine and the progression through its states.

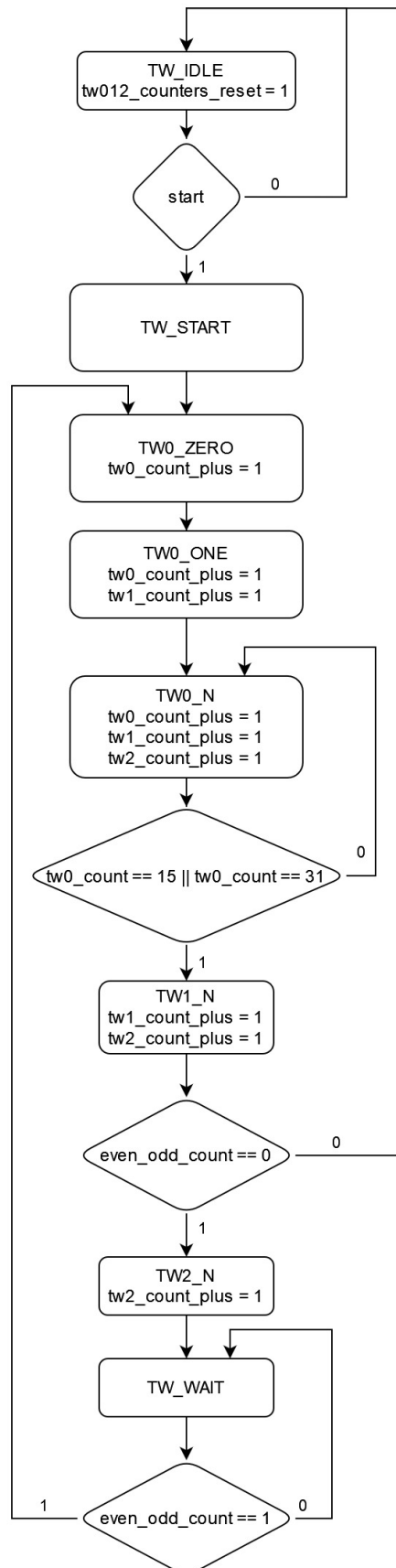


FIGURE 8: Flowchart showing the control unit's starter section twiddle factor state machine and the progression through its states.

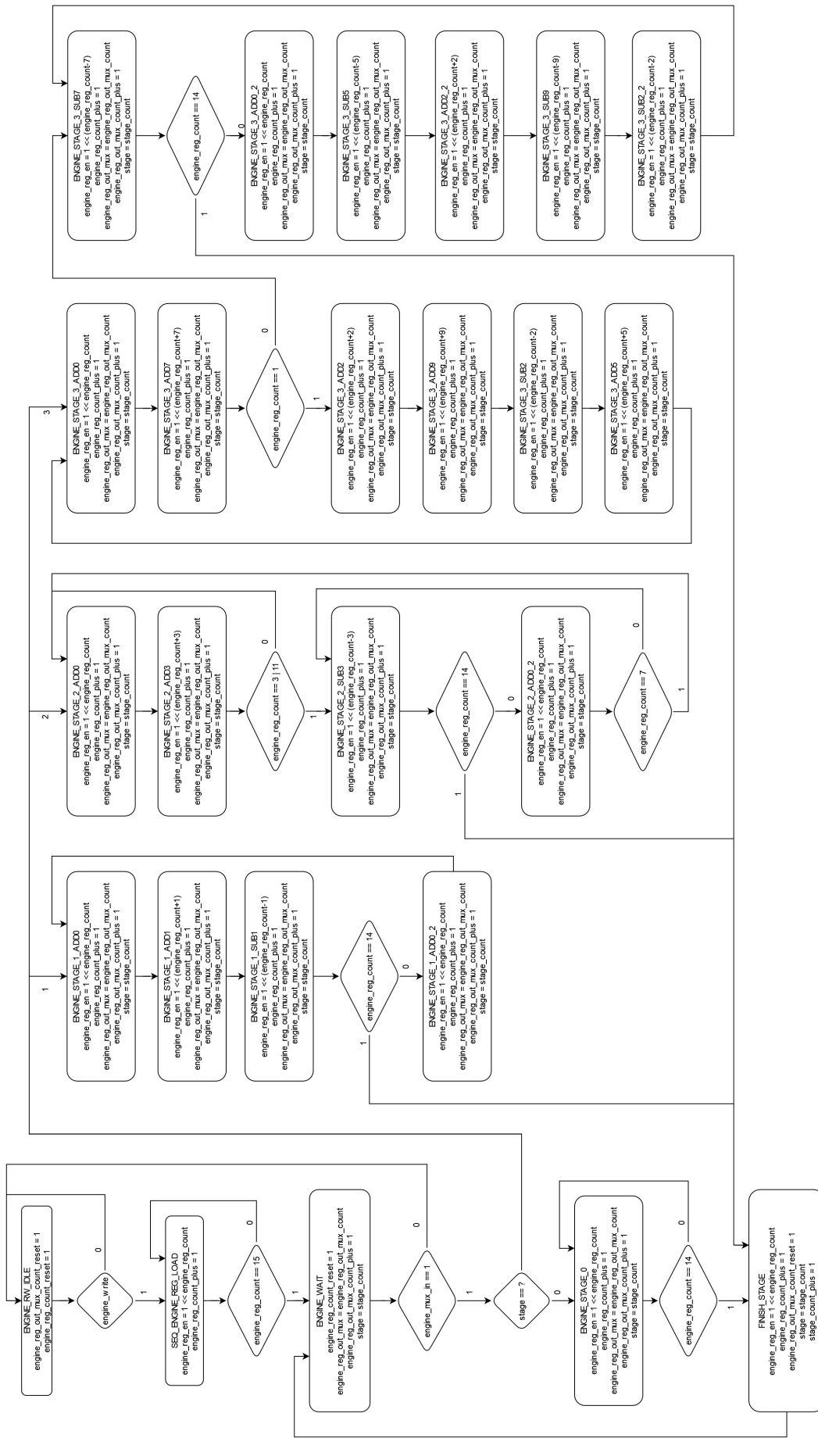


FIGURE 9: Flowchart showing the control unit's engine memory access state machine and the progression through its states.

Appendix C - Design Archive

This appendix provides a brief table of contents for the submitted design archive .zip file.

1. SystemVerilog Hardware Code
2. Testing
 - Testbenches
 - Modelsim Simulations
 - BU
 - PIPE_ENGINE
 - FULL_PIPE
 - CONTROL
 - Software Reference Programs
3. State Machine Flowcharts
4. Quartus Projects
 - 256_engine1
 - 256_engine1_C10GX
 - 256_engine1_S10GX
5. Auxiliary Software Programs

Appendix D - Original Project Brief

Project Brief:

Project Title: *Hardware Acceleration of Number Theoretic Transform Operations for Post-Quantum Cryptography*

Student Name: Richard Karlson

Supervisor: Tomasz Kazmierski

Project Description:

Current cryptography methods keep data and devices secure against most threats, however advances in quantum computing are forcing cryptography into a post-quantum era. New encryption and decryption methods are more resilient to quantum computers but introduce additional computational demands. The multiplication of polynomials using Number Theoretic Transforms (NTT) is one of these new demands that is used by prominent post-quantum cryptography schemes. Unfortunately, this process is resource intensive which slows down encryption and decryption processes and particularly affects resource-constrained environments.

A solution is a dedicated hardware accelerator for polynomial multiplication and NTT operations. These operations are suitable for parallel execution and require unique mathematical calculations which makes specialised hardware an appealing option. Through this project, I would like to investigate performance improvements, such as speed and energy efficiency, that a hardware accelerator would have on NTT operations when compared to an equivalent software implementation.

Goals:

- Synthesise a working hardware accelerator for NTT operations onto an FPGA using SystemVerilog.
- Write a test case program for an ARM core in the FPGA to utilise the hardware accelerator.
- Write an equivalent software implementation to run on the ARM core without the accelerator.
- Compare the performance between the accelerated and non-accelerated implementations, showing any performance improvements.

Stretch Goals:

- Implementation of a full encryption and decryption test case with a post-quantum cryptography scheme that utilises NTT and the accelerator.